Expert Systems With Applications 300 (2026) 130392

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

CIV: Leveraging causal subgraphs of vulnerability for noise reduction in vulnerability detection

Zhenyu Gao [a,1], Lei Xiao [a,1,*], Xia Du [a], Ying Xing [b]

[a] School of Computer and Information Engineering, Xiamen University of Technology, Xiamen, 361024, China
[b] School of Intelligent Engineering and Automation, Beijing University of Posts and Telecommunications, Beijing, 100876, China

ARTICLE INFO

Keywords:
Source code vulnerability detection
Graph neural network
Software security
Deep learning

ABSTRACT

Accurate vulnerability detection is critical for software security. Although deep learning-based vulnerability detection methods have shown promise in this task, they include much information unrelated to vulnerability semantics, which we call noise. Noise disrupts model learning and may cause the model to learn spurious vulnerability features. To address this challenge, we propose CIV, a novel and causality-aware vulnerability detection framework that explicitly focuses on the code semantics that cause vulnerability. The key novelty of CIV lies in introducing a novel slice representation: causal subgraphs of vulnerability. CIV characterizes vulnerability from a causal perspective by capturing the subgraphs related to vulnerability semantics within the code property graph. By extracting these causal subgraphs, CIV effectively reduces noise and focuses on learning meaningful vulnerability features. Therefore, we design EApooling, a novel graph pooling layer, to adaptively extract these causal subgraphs. However, since EApooling is influenced by the predefined pooling rate, some noisy nodes may remain in the extracted subgraphs. To mitigate this issue, we adopt DIFFormer, which utilizes an energy-constrained diffusion mechanism to update node features. This diffusion mechanism maintains low semantic affinity between noisy and non-noisy nodes, effectively limiting the spread of noisy node features to non-noisy nodes, thereby alleviating the impact of the remaining noisy nodes on the detection results. We evaluate CIV on two real-world vulnerability datasets: NVD and GITA. Experimental results show that CIV outperforms previous state-of-the-art methods, achieving F1 score of 87.30 % on the NVD dataset and 82.39 % on the GITA dataset, with improvements of 4.38 % and 1.56 % over the best prior results, respectively.

1. Introduction

Software security is becoming critical in today's fast-evolving development landscape. In 2024, a record 40,301 common vulnerabilities and exposures (CVEs) were reported,[2] marking a 38.6 % increase over the previous year. To address the growing threat of software vulnerabilities, deep learning-based detection methods have become a mainstream research direction (Ma et al., 2025; Pang et al., 2025; Shao et al., 2025; Tang et al., 2024; Tian et al., 2024; Wu et al., 2023b; Yu et al., 2025; Zhang et al., 2025).

Current deep learning-based vulnerability detection methods can be broadly categorized into function-based and slice-based approaches.

Function-based vulnerability detection methods treat the entire function as the basic analysis unit and construct structural or semantic representations to learn vulnerability patterns. Some methods leverage control flow graph (CFG) and data flow graph (DFG) to represent execution semantics, such as those proposed by Wang et al. (2020), Steenhoek et al. (2024), and Yuan et al. (2023). Others analyze the syntactic structure of source code by leveraging abstract syntax tree (AST), as seen in Zhou et al. (2019), Luo et al. (2022), and Tian et al. (2024). Some approaches further model functions as token sequences or natural language-like inputs, including Zhang et al. (2024) and Lu et al. (2024). Alternative representations include modeling code as images (Wu et al., 2022b). Despite these differences, all these methods share the common strategy of learning vulnerability patterns at the function level, which inevitably introduces much noise. This noise limits their detection accuracy. Slice-based methods have been proposed to reduce noise by focusing on finer-grained program representations to address this problem. Slice-based vulnerability detection methods typically operate on slices extracted via control and data dependencies, enabling finer-grained analysis than function-based methods. Some approaches first construct program graphs, such as program dependency graph (PDG) or

* Corresponding author.
E-mail addresses: zygao@stu.xmut.edu.cn (Z. Gao), lxiao@xmut.edu.cn (L. Xiao), duxia@xmut.edu.cn (X. Du), xingying@bupt.edu.cn (Y. Xing).
[1] The two authors contribute equally to this work.
[2] https://www.cvedetails.com/

https://doi.org/10.1016/j.eswa.2025.130392
Received 28 July 2025; Received in revised form 18 October 2025; Accepted 9 November 2025
Available online 17 November 2025

inter-procedural graph (IG), and then perform slicing over them to encode structural dependencies (Li et al., 2021b, 2018; Shao et al., 2025; Wu et al., 2023a; Xiao et al., 2024). Others utilize slicing to isolate critical paths or statements for subgraph learning or statement-level localization (Cheng et al., 2024; Li et al., 2021a; Mirsky et al., 2023). In addition, some methods adopt contrastive learning to prioritize semantically relevant slices for vulnerability detection (Cheng et al., 2022).

However, existing slice-based detection methods still face a severe noise problem, as most slicing algorithms are fully manually designed and tend to include all nodes that are reachable from the slicing entry node. As a result, these methods often fail to remove noisy nodes connected to non-noisy ones, leading to the retention of many noisy nodes. These noisy nodes can mislead the model into learning spurious discriminative patterns, leading to severe overfitting. For example, suppose many training samples containing the statement tmp = 0 are labeled as vulnerable, even though this statement has no relation to the vulnerability semantics. In that case, the model may incorrectly learn to associate tmp = 0 with the presence of a vulnerability. Such superficial co-occurrence phenomena can cause the model to focus on meaningless structural cues, rather than the underlying vulnerability mechanisms, ultimately degrading the model's robustness in real-world scenarios. Therefore, a new detection method is urgently needed to learn to differentiate between noisy and non-noisy nodes and capture vulnerability patterns through the interactions among non-noisy nodes.

To this end, we propose CIV, a novel and causality-aware vulnerability detection framework that explicitly focuses on the code semantics that cause vulnerability. CIV first extracts property graph-based vulnerability candidates (PrVCs) from the code property graph (CPG), with PrVCs being a slice representation proposed by GraphFVD (Shao et al., 2025). On this basis, we design EApooling, a novel graph pooling layer that adaptively extracts the causal subgraphs of vulnerability—a new slice representation proposed in this work. Causal subgraphs of vulnerability refer to the subgraphs that consistently indicate the presence of a vulnerability, regardless of variations in the surrounding graph structure. By extracting these causal subgraphs, CIV effectively reduces noise, enabling the model to focus on learning meaningful vulnerability features from a causal perspective. However, since EApooling calculates edge attention and selects edges to retain based on a predefined pooling rate, some noisy nodes may remain in the extracted subgraphs. To mitigate this issue, we adopt DIFFormer (Wu et al., 2023c), which utilizes an energy-constrained diffusion mechanism to update node features. This diffusion mechanism maintains low semantic affinity between noisy and non-noisy nodes, effectively limiting the spread of noisy node features to non-noisy nodes, thereby alleviating the impact of the remaining noisy nodes on the detection results. Finally, the features of the nodes in the causal subgraphs are aggregated to obtain the final graph representation. A linear layer is then applied over the graph representation to predict whether the PrVC contains a vulnerability.

We evaluate the effectiveness of CIV using two real-world vulnerability datasets, comparing it with state-of-the-art vulnerability detection approaches, and find that CIV outperforms existing methods, achieving higher detection accuracy and better generalization. Additionally, we examine the performance of DIFFormer in comparison to other graph encoders and observe that DIFFormer excels in reducing the interference from noisy nodes, leading to more precise vulnerability detection. We also investigate the effectiveness of EApooling in enhancing the extraction of causal subgraphs, comparing it with existing pooling methods, and confirm that EApooling enhances overall performance. We further analyze the contribution of each component within CIV, exploring its impact on the overall detection performance, and conclude that each component plays a crucial role. Finally, we evaluate the generalization capability of CIV under strict dataset splits and cross-project scenarios. The results show that although CIV performs better than previous methods, there is still significant room for improvement in these challenging settings.

To summarize, our paper makes the following contributions:

- We propose a novel slice representation, called the causal subgraphs of vulnerability, which characterizes vulnerability from a causal perspective. To effectively extract these causal subgraphs, we design EApooling, a novel graph pooling layer that calculates edge attention to adaptively identify and retain the most causally relevant substructures.
- We propose CIV, a novel and causality-aware vulnerability detection framework that explicitly focuses on the code semantics that cause vulnerability. CIV adopts DIFFormer, which employs an energy-constrained diffusion mechanism to update node features. This diffusion mechanism effectively reduces the impact of the remaining noisy nodes in the causal subgraphs of vulnerability on the detection results.
- We evaluate the effectiveness of our approach, CIV, using two real-world vulnerability datasets. The experimental results show that CIV achieves consistently better performance and stronger generalization than existing vulnerability detection methods.
- We have publicly released the implementation of CIV to facilitate reproducibility and future research. The source code is available in our open repository at https://github.com/boomboomgzy/CIV.

2. Related work

In this section, we review existing research on deep learning-based vulnerability detection to position our work within the broader context of prior studies. Current deep learning-based vulnerability detection methods can be broadly classified into function-based and slice-based approaches. Function-based methods generally treat whole functions as the basic unit of analysis, constructing representations that capture the function's overall structure and semantics. In contrast, slice-based methods focus on more granular program segments by extracting slices derived from control and data dependencies, enabling a more focused analysis of potentially vulnerable regions.

Early function-based methods, such as Devign (Zhou et al., 2019), encoded functions as graphs by augmenting AST with control and data flow information. However, by abstracting data flow into only three coarse-grained edge types, Devign loses fine-grained semantic information necessary for modeling complex data dependencies. To improve semantic expressiveness, ReGVD (Nguyen et al., 2022) linearized code into token sequences before constructing graphs, better preserving lexical and contextual order. This, though, weakened structural dependencies. Yuan et al. (2023) constructed function behavior graphs to encode abstract functional intents, but such abstraction risks overlooking concrete code-level indicators of vulnerability. Similarly, DEEPVD (Wang et al., 2023) integrated features from multiple abstraction layers, including post-dominance and exception-flow graphs, to enhance semantic alignment. Nevertheless, this increased model complexity and introduced feature redundancy. To refine the structural granularity of representations, TrVD (Tian et al., 2024) decomposed AST into ordered subtrees, capturing fine-grained syntax while risking the loss of global semantic coherence. CAG (Luo et al., 2022) proposed a compact AST representation to reduce graph size and improve learning efficiency, though it discarded control and data dependencies, limiting its semantic completeness. To enrich contextual information, Steenhoek et al. (2024) incorporated inter-procedural knowledge into function-level graphs, which came at the cost of increased graph complexity and reduced scalability. In another direction, Wu et al. (2022b) transformed function code into images and applied convolutional neural network (CNN) for vulnerability detection. This visual approach, unfortunately, sacrifices structural and semantic precision. LineVul (Fu & Tantithamthavorn, 2022) leveraged transformer architectures over statement sequences to capture long-range dependencies from textual context but lacked explicit structural guidance from syntax or program graphs. Finally, VulSim (Shimmi et al., 2024) fused multiple pre-trained models from different domains to capture diverse semantic cues. Even so, its reliance on decision trees hindered its deep learning optimization capabilities. In

recent years, large language models (LLMs) have attracted considerable interest for their powerful capabilities, prompting their exploration in vulnerability detection (Ding et al., 2025; Mechri et al., 2025; Sheng et al., 2025; Shestov et al., 2025; Zhou et al., 2025). Lu et al. (2024) proposed GRACE, a detection framework that integrates graph-based code representations with LLM-driven contextual understanding to improve vulnerability identification. Yet, these methods heavily rely on prompt engineering or fine-tuning and may struggle to align general language understanding with code-specific semantics.

VulDeePecker exemplified early efforts in slice-based vulnerability detection (Li et al., 2018), which constructed slices using code gadgets formed around key application programming interface (API) calls. SySeVR (Li et al., 2021b) extended this idea by identifying vulnerability-triggering tokens to generate more targeted semantic slices, while VulDeeLocator (Li et al., 2021a) further incorporated attention mechanisms to highlight important code segments. However, all these methods extracted slices at the granularity of code lines, making them prone to include substantial redundant or irrelevant code, thereby diluting semantic precision. To improve the semantic relevance of slices, Guo et al. (2022) proposed a hybrid framework that fused taint-based slicing with token-level modeling. By leveraging taint propagation chains, the method better isolates vulnerability-related paths. Similarly, Salimi and Kharrazi (Salimi & Kharrazi, 2022) applied information flow analysis to extract semantically meaningful regions, enhancing data dependency coverage. These methods still relied on static heuristics and often failed to model control-flow information. Dong et al. (2023) introduced SedSVD, which builds relational subgraphs around each statement using CPG and applies relational graph convolutional network (RGCN) to learn semantic dependencies. Yet, the subgraph extraction at the statement level still introduced irrelevant nodes, affecting detection precision. To explicitly capture structural dependencies, SnapVuln (Wu et al., 2023a) constructed IG and extracted slices from it to account for cross-functional control and data dependencies. Still, the model did not consider syntactic structures of code, limiting representation richness. To address this limitation, EnGS2F (Xiao et al., 2024) constructed a context relationship graph (CRG) from PDG and applied gated graph neural network (GGNN) for representation learning. Despite incorporating diverse dependency types, the relational graph was treated as multiple untyped directed edges, weakening its ability to model relation-specific semantics. Cheng et al. (2024) further refined the slicing process by extracting critical vulnerability-triggering paths and constructing subgraphs from CPG, thus offering a more focused vulnerability context. Even so, the effectiveness of this approach hinges on the precision of path extraction. Previous methods cannot generally precisely pinpoint vulnerable statements. To address this limitation, Wattanakriengkrai et al. (2020) applied local interpretable model-agnostic explanation (LIME) to highlight risky tokens and subsequently classified the containing statements as vulnerable. Nevertheless, LIME relies on post-hoc interpretation rather than being integrated into the learning process, making it unstable and sensitive to model perturbations. Similarly, DeepLineDP (Pornprasit & Tantithamthavorn, 2022) introduced granularity refinement modules based on attention mechanisms following recurrent neural network (RNN) architectures to localize vulnerable statements. Still, these methods lacked explicit modeling of token-level dependencies and were prone to attention drift, which reduced the accuracy of statement-level localization.

Overall, current vulnerability detection methods suffer from severe noise, limiting their ability to capture vulnerability semantics. Function-based methods tend to encode large amounts of irrelevant contextual information. In contrast, slice-based methods rely on fully manually designed slicing algorithms that usually include all nodes reachable from slicing entry points (SEPs), inevitably retaining many noisy nodes. Consequently, existing approaches often learn spurious correlations rather than actual vulnerability patterns, which reduces their effectiveness.

To overcome the limitation caused by noise, our work adopts a causal perspective on slice representation. Compared to prior slice-based approaches such as SySeVR (Li et al., 2021b), SedSVD (Dong et al., 2023), and GraphFVD (Shao et al., 2025), which rely on fully manually defined slicing rules and passively include all reachable nodes, CIV employs the EApooling module to adaptively identify and extract the causal subgraphs of vulnerability, focusing on code regions that are causally related to vulnerability and thereby mitigating the excessive inclusion of noisy nodes. Moreover, by integrating DIFFormer's energy-constrained diffusion mechanism, CIV explicitly suppresses the propagation of residual noise, a capability absent in most prior slice-based methods. Nevertheless, CIV still inherits certain limitations from this research line: its performance partially depends on the slicing quality, and the predefined pooling ratio in EApooling may restrict the precision of causal subgraph extraction. Future work could address these issues by exploring dynamic pooling mechanisms or end-to-end graph generation frameworks.

```
1 unsigned long perf_instruction_pointer(struct
      pt_regs *regs)
2 {
3   bool use_siar = regs_use_siar(regs);
4   unsigned long siar = mfspr(SPRN_SIAR);
5
6   if (ppmu->flags & PPMU_P10_DD1) {
7     if (siar)
8       return siar;
9     else
10      return regs->nip;
11  } else if (use_siar && siar_valid(regs))
12    return mfspr(SPRN_SIAR) + perf_ip_adjust(regs);
13  else if (use_siar)
14    return 0;    // no valid instruction pointer
15  else
16    return regs->nip;
17 }
```

Code 1. An example of the vulnerable code (CVE-2021-38200). The code on line 6 directly accesses ppmu->flags without verifying whether ppmu is initialized.

3. Preliminaries

In this section, we introduce the causal subgraphs of vulnerability and DIFFormer. The former represents a new way of characterizing vulnerability from a causal perspective, aiming to capture the essential code structures that directly lead to vulnerability. The latter, DIFFormer, provides a diffusion-based mechanism for feature propagation, designed to suppress the influence of noise and enhance the semantic purity of learned representations.

3.1. Causal subgraphs of vulnerability

Following the causal perspective proposed by Wu et al. (2022a), we model the vulnerability detection task using an SCM, where each PrVC is represented as an input graph G composed of two disjoint components: the causal subgraphs C and the non-causal subgraphs S, as illustrated in Fig. 2. The causal subgraphs C contain the minimal code elements that directly cause the vulnerability label Y, while the non-causal subgraphs S may be spuriously correlated with Y due to confounding, contextual bias, or dataset artifacts. In this model, the vulnerability label Y is determined solely by the causal part C, and the non-causal part S has no additional effect on Y once C is given. This relationship can be formally expressed as:

$$Y = f(C), \quad Y \perp S \mid C \tag{1}$$

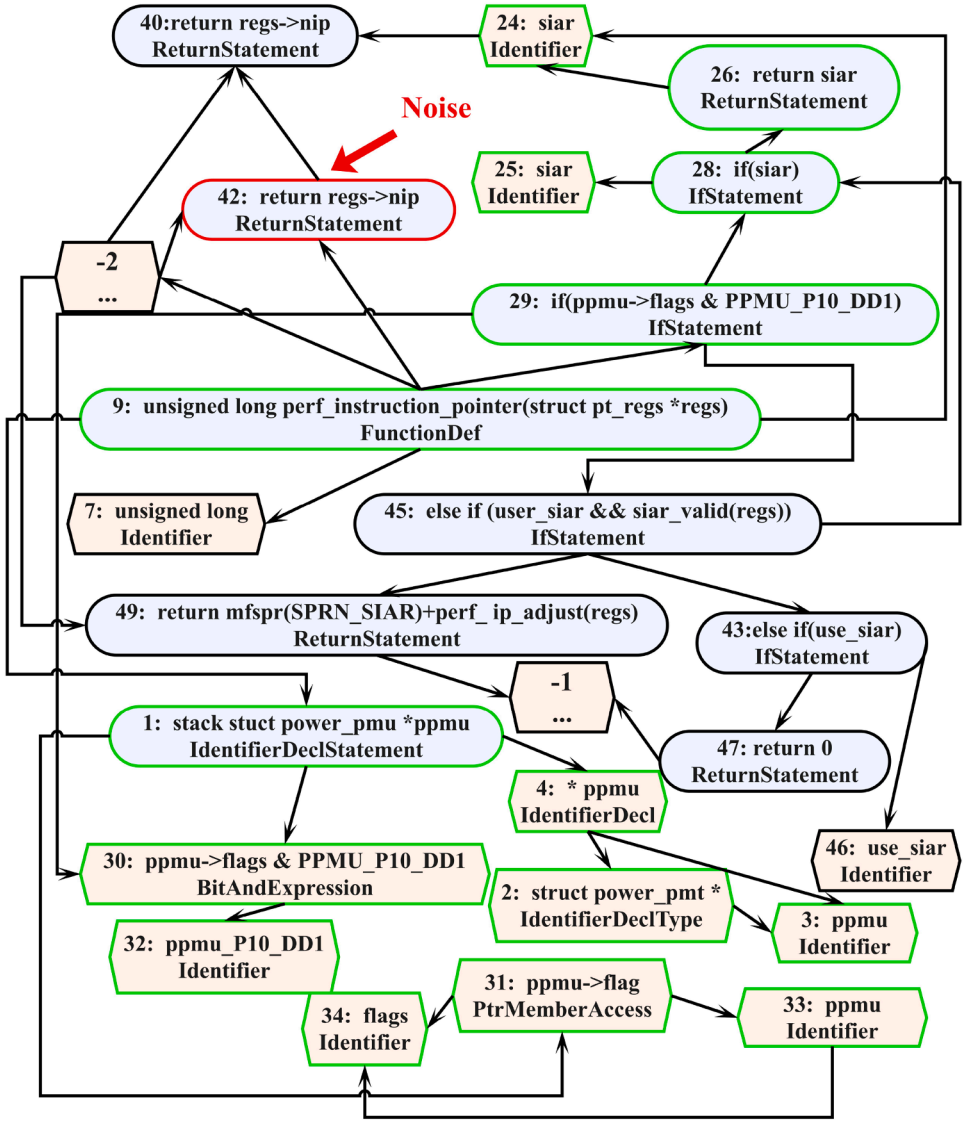

Fig. 1. A PrVC extracted from Code 1. Each blue box denotes a statement in the source code, and each orange box represents an AST node. All edges are shown as black solid lines, indicating that our method does not depend on edge-type information. Clearly, the PrVC contains considerable noise-for instance, node 42 is unrelated to the vulnerability and serves as a representative noisy node. Nodes outlined in green indicate a subgraph sufficient for identifying this vulnerability, which serves as an example of the causal subgraph of the vulnerability. For clarity, several AST nodes that are not essential for understanding the vulnerability have been merged into single nodes (e.g., node −1 and node −2).

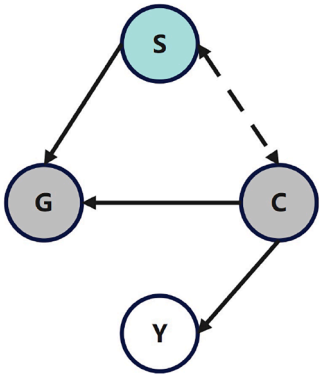

Fig. 2. Structural causal model (SCM). SCM consists of four key components: S, C, G, and Y. S represents the non-causal subgraphs, C is the causal subgraphs, G is the input graph combining both S and C, and Y represents the graph label.

Building on this causal perspective, we define the causal subgraphs of vulnerability as the subgraphs that consistently indicate the presence of a vulnerability, regardless of variations in the surrounding graph structure. Therefore, instead of relying on the entire PrVC, which may contain irrelevant or misleading information from the non-causal subgraphs S, we focus on extracting and learning from only the causal subgraphs C. This approach enhances the model's ability to detect vulnerability accurately and reduces the influence of spurious correlations.

Here we give an example of the causal subgraphs of vulnerability. First, consider the code shown in Code 1, highlighting a null pointer dereference vulnerability. If a platform-specific PMU driver is not registered, the pointer 'ppmu' is null. However, the code on line 6 accesses 'ppmu->flags' directly without verifying whether 'ppmu' has been properly initialized. This oversight can lead to a runtime crash when the function is executed on unsupported platforms. A proper fix would be to insert a null check before dereferencing 'ppmu', preventing invalid memory access. Fig. 1 presents a PrVC extracted from Code 1. While the PrVC captures a wide range of dependencies, only a small subset

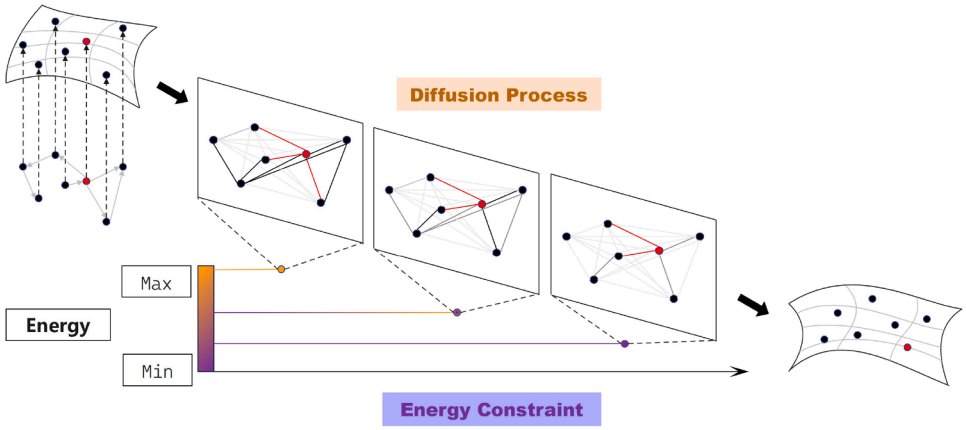

Fig. 3. The general concept of DIFFormer. DIFFormer takes a batch of instances as input and encodes them into hidden states through a diffusion process aimed at minimizing an energy constraint.

is relevant to the vulnerability semantics. The null pointer dereference occurs when 'ppmu' is accessed without a null check, as illustrated at node 29 ('ppmu->flags').Nodes outlined in green indicate a subgraph sufficient for identifying this vulnerability. Specifically, it captures the declaration of the pointer 'ppmu' and its direct use in the conditional expression 'ppmu->flags' without any prior null check. Since this subgraph represents unsafe access where the pointer is used without verifying its validity, analyzing this subgraph alone is sufficient to detect the vulnerability, without requiring additional information from the rest of the graph. Therefore, we consider this subgraph as an example of the causal subgraphs of the vulnerability.

3.2. DIFFormer

As shown in Fig. 4, CIV adopts DIFFormer in both the slice extraction and vulnerability detection stages to update node features. Among the various graph encoders available, we choose DIFFormer for its energy-constrained diffusion mechanism, which simultaneously considers structural connections and semantic affinity between nodes. DIFFormer effectively maintains low semantic affinity between noisy and non-noisy nodes, thus limiting the spread of noisy node features to non-noisy nodes. This characteristic helps reduce the interference from noisy nodes, enhancing the model's ability to capture actual vulnerability patterns.

Each DIFFormer layer updates node features through energy-constrained diffusion mechanism, as illustrated in Fig. 3. Specifically, node features are iteratively updated as:

$$\mathbf{z}_i^{(k+1)} = \left(1 - \frac{\tau}{2}\sum_{j=1}^{n}\left(\hat{S}_{ij}^{(k)} + \tilde{A}_{ij}\right)\right)\mathbf{z}_i^{(k)} + \frac{\tau}{2}\sum_{j=1}^{n}\left(\hat{S}_{ij}^{(k)} + \tilde{A}_{ij}\right)\mathbf{z}_j^{(k)} \tag{2}$$

where $\hat{S}_{ij}^{(k)}$ represents the semantic affinity between nodes i and j and $\tilde{A}_{ij} = \frac{1}{\sqrt{d_i d_j}}$ if $(i,j) \in E$, and 0 otherwise, which represents the structural connections between nodes i and j. Here, d_i is the degree of node i.

The following energy minimization objective drives this diffusion process:

$$E(Z,k;\delta) = \|Z - Z^{(k)}\|_F^2 + \frac{\lambda}{2}\sum_{i,j}\delta\left(\|\mathbf{z}_i - \mathbf{z}_j\|_2^2\right) + \frac{\lambda}{2}\sum_{(i,j)\in E}\|\mathbf{z}_i - \mathbf{z}_j\|_2^2 \tag{3}$$

where the first term encourages temporal consistency, the second promotes semantic alignment, and the third enforces smoothness over the graph topology. The function $\delta : \mathbb{R}^+ \to \mathbb{R}$ is *non-decreasing* and *concave*, encouraging close node pairs to stay close while limiting over-smoothing.

The semantic affinity $\hat{S}_{ij}^{(k)}$ are computed as the derivative of function δ, instantiated by the sigmoid function:

$$\hat{S}_{ij}^{(k)} = \frac{1}{1 + \exp\left(-(\mathbf{z}_i^{(k)})^\top \mathbf{z}_j^{(k)}\right)} \tag{4}$$

the sigmoid function ensures non-negativity, monotonicity, and saturation, which are desirable properties for stabilizing diffusion and preserving smooth gradients during training. Compared to the dot-product kernel, the sigmoid kernel better handles high inner-product magnitudes and avoids over-sharpened attention, especially in slices where node features vary widely.

4. Design

In this section, we present the proposed vulnerability detection framework, CIV. We first outline its overall architecture and then describe each stage in detail. As illustrated in Fig. 4, CIV extracts causal subgraphs of vulnerability and adopts DIFFormer for detecting vulnerability. The framework comprises three stages: 1) Construct the CPG from source code, integrating abstract syntax, control flow, and data flow to capture comprehensive program semantics. 2) Select SEPs from the CPG to extract PrVCs. The extracted PrVCs are first fed into DIFFormer, which employs an energy-constrained diffusion mechanism to refine node representations and suppress the propagation of noise. This process yields noise-reduced contextual embeddings that preserve essential vulnerability semantics. Then, EApooling is applied to adaptively extract the causal subgraphs of vulnerability, providing finer-grained representations directly related to vulnerability semantic. 3) The other DIFFormer is further applied to re-encode the updated context and mitigate the influence of residual noise within the causal subgraphs. The resulting graph-level representations are then passed to a classification module, which determines whether each PrVC contains a vulnerability, enabling accurate and robust detection.

4.1. CPG construction

We construct the CPG using Joern,[3] a widely adopted static analysis tool for program graph generation. Given the source code as input, Joern first parses the code into an AST, which captures the syntactic structure of the program. It then generates the CFG to describe the possible execution paths within each function, and the DFG tracks the data flow between variable definitions and their corresponding uses. The CPG is

[3] https://joern.io/

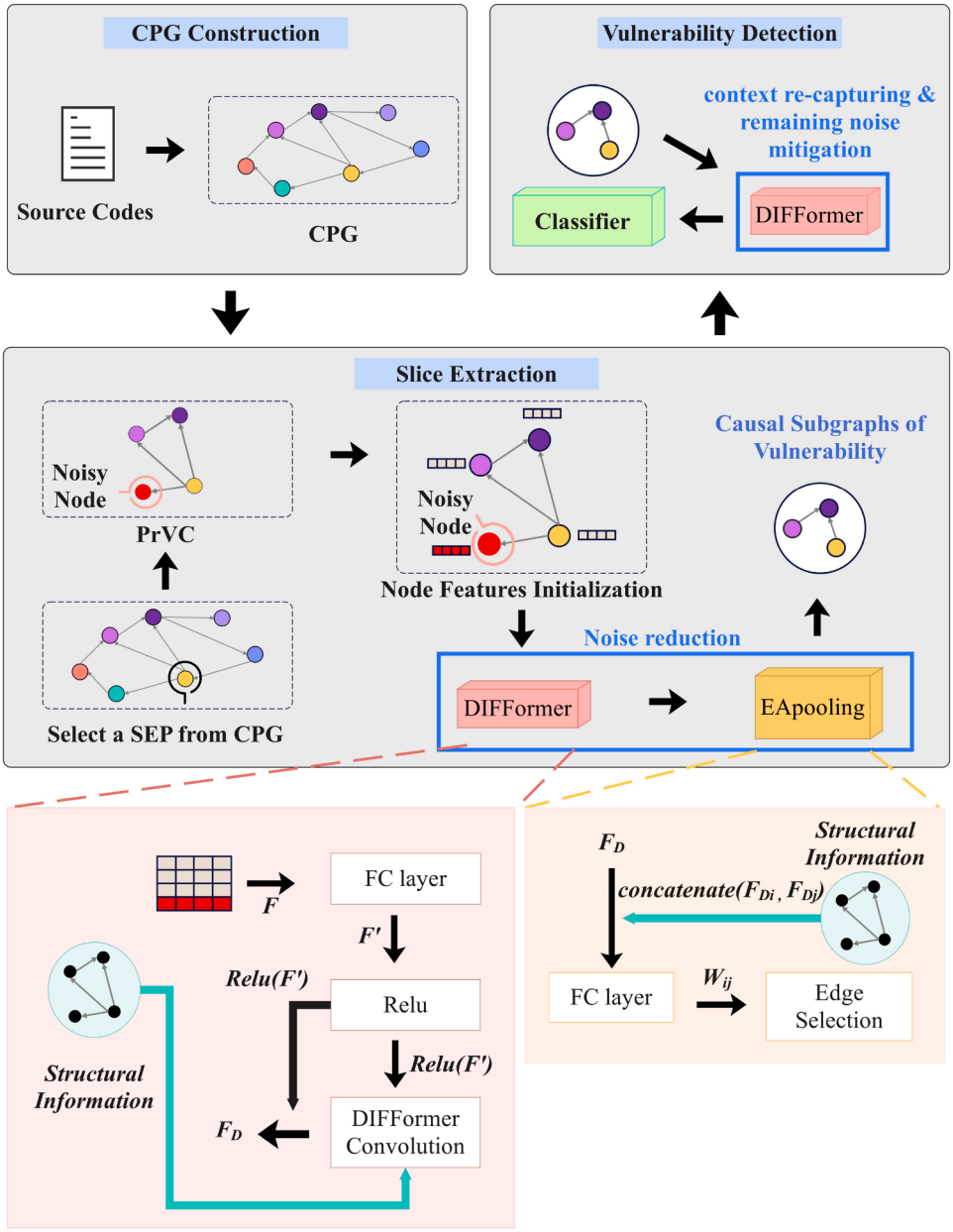

Fig. 4. Overview of CIV. Extracting causal subgraphs of vulnerability relies on two core components: DIFFormer and EApooling. The detailed algorithms of these two components will be described in Section 4.2.3. Since subgraph extraction inevitably alters the original neighborhood structure of nodes, a second DIFFormer layer is applied to re-encode the updated context and mitigate the influence of remaining noise within the causal subgraph.

formed by combining the AST, CFG, and DFG into a unified graph representation, where the nodes are shared across all three graphs, enabling cross-graph traversal.

The CPG integrates syntactic and semantic information, including statement-level operations, control flow dependencies, and data flow relationships. This unified graph structure is the foundation for our subsequent slicing and vulnerability detection. We store the CPG in the Neo4j graph database to support efficient querying and large-scale processing.

Our framework deliberately does not rely on edge-type information from the CPG. It is because DIFFormer considers the semantic affinity between any two nodes. Different edge types might require calculating distinct semantic affinities if we integrated edge-type information. This would undoubtedly increase the model's complexity and computational cost.

4.2. Slice extraction

4.2.1. PrVCs extraction

Following the SEP selection strategy used in GraphFVD (Shao et al., 2025), we identify several categories of program elements as potential vulnerability indicators. Sensitive variables—such as pointers and arrays—are significant sources of program vulnerability, including null pointer dereference, use-after-free, and buffer overflow. These variables are commonly involved in unsafe memory access patterns, such as dereferencing a null pointer (*ptr) or accessing structure members through potentially uninitialized objects (ptr->field). Such operations can lead to critical failures or exploitable security issues if not correctly validated. Therefore, we treat expressions involving pointer dereference and structure field access as SEP. Traditional static analysis tools also support

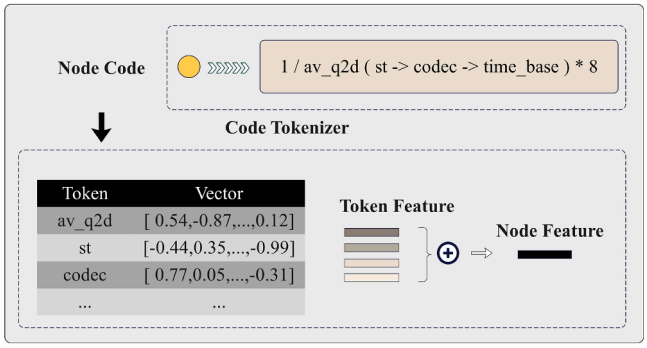

Fig. 5. The process of node feature initialization.

this design choice, which widely uses pointers and array variables as core features for detecting memory-related vulnerability. In addition, we consider library/API function calls to be important SEPs. Many standard library functions—memcpy, strcpy, memset, and malloc—are frequently misused and consistently reported by static vulnerability scanners as familiar sources of memory corruption and resource management issues. We collect a total of 1506 commonly misused functions. Furthermore, arithmetic expressions are critical in program logic and data flow and are often associated with integer overflow, underflow, and division-by-zero vulnerability. As a result, statements that involve arithmetic operators such as '+', '-', '*', and '/' are also selected as SEPs. Lastly, we include if-condition statements as SEPs because they often guard sensitive operations and control whether a vulnerability can be triggered. For example, missing or incorrect null checks in condition expressions can lead to null pointer dereference or other conditional bugs. Including such control-flow predicates helps capture vulnerability-related decision logic.

Specifically, we first identify SEPs based on two node-level attributes in the CPG: type and code. We apply lightweight pattern-matching rules to these attributes to extract four categories of SEPs commonly associated with vulnerability. Specifically, a node is selected as a SEP if it satisfies any of the following conditions: (1) its type is Function, and its code matches one of 1506 collected API/library function calls; (2) its type is IdentifierDeclStatement or Parameter, and the code contains '=' along with either '*' or '['; (3) its type is ExpressionStatement and the code contains '=' and matches an arithmetic expression; (4) its type is IfStatement or Parameter, and the code contains a conditional expression such as if.

After identifying SEPs, we apply slicing to extract PrVCs from the CPG. The slicing process begins by selecting a SEP and iteratively expanding a reachable node set. At each step, we include nodes connected to any node already in the reachable set, regardless of edge direction. For example, in the PrVC shown in Fig. 1, starting from Node 1, we identify that Nodes 4, 9, 30, and 31 are directly connected to Node 1 and should therefore be added to the reachable node set. The traversal then recursively traverses from these newly added nodes, expanding the PrVC until no additional connected nodes are found.

Compared to the entire CPG, the resulting PrVCs contain significantly fewer nodes, especially in large functions with many lines of code. Then, we label each PrVC based on whether it contains any vulnerable statements. Specifically, if at least one node in the PrVC corresponds to a known vulnerable statement in the source code, the PrVC is labeled as '1' (vulnerable); otherwise, it is labeled as '0' (non-vulnerable). As illustrated in Fig. 1, the PrVC is labeled as '1' because it includes node 29 that matches the vulnerable statement 'if (ppmu->flags & PPMU_P10_DD1)'.

4.2.2. Node feature initialization

Here, we provide a detailed explanation of how we initialize the feature of the nodes in PrVCs. As illustrated in Fig. 5, each node corresponds to a sequence of tokens $\{t_1, t_2, \ldots, t_n\}$, where each t_i is obtained using the tokenizer of GraphCodeBERT (Guo et al., 2020), a pre-trained model that leverages both code syntax and data flow information during its pretraining. This allows GraphCodeBERT to generate highly semantic and context-sensitive token features, making it particularly suitable for capturing the structural and functional relationships within code. Each token is then mapped to a pre-trained feature $\mathbf{e}_{t_i} \in \mathbb{R}^d$. The final feature of the node $\mathbf{h}_v$ is computed by summing the features of all its tokens:

$$\mathbf{h}_v = \sum_{i=1}^{n} \mathbf{e}_{t_i} \tag{5}$$

This approach allows us to initialize node features with semantically rich token features derived from GraphCodeBERT, ensuring consistency with the underlying code semantics.

Algorithm 1 DIFFormer convolution.

Input: node features $X \in \mathbb{R}^{N\times F}$, adjacency matrix A_{graph}.
Output: node outputs $Y \in \mathbb{R}^{N\times d}$.
$Q \leftarrow XW_q$, $K \leftarrow XW_k$, $V \leftarrow XW_v$ ▷ project to $[N, d]$
$\hat{S}_{ij} = \dfrac{1}{1+\exp\left(-Q_i^{\top}K_j\right)}$ ▷ sigmoid semantic affinity
$A_{ij} \leftarrow \dfrac{\hat{S}_{ij}}{\sum_{j'=1}^{N}\hat{S}_{ij'}}$ ▷ normalization
$Z_{\text{attn}} \leftarrow AV$ ▷ semantic-aware aggregation, shape $[N, d]$
$\tilde{A} \leftarrow D^{-1/2}A_{\text{graph}}D^{-1/2}$, $Z_{\text{gcn}} \leftarrow \tilde{A}V$ ▷ graph convolutional network
$Z \leftarrow Z_{\text{attn}} + Z_{\text{gcn}}$
$Y \leftarrow Z$
return Y

4.2.3. Subgraphs extraction

Given an input PrVC with initialized node features, we pass it through DIFFormer, which updates the node features based on the structural relationships and semantic affinities between nodes. Algorithm 1 shows the core layer of DIFFormer, the **DIFFormer Convolution**. It first computes semantic affinities between nodes and performs semantic-aware aggregation based on these affinities (Lines 3–6). Then it uses a graph convolutional network (GCN) to learn structural relationships from the input graph (Line 7). The node features produced by the semantic aggregation and the GCN are summed to yield the final node features (Lines 8–9). This allows DIFFormer to capture the contextual semantics of each node, laying the foundation for subsequent subgraph extraction.

Algorithm 2 EApooling.

Input: node features $X \in \mathbb{R}^{N\times F}$, edge index $edge\_index$, ratio r
Output: causal subgraphs of vulnerability
$(start, end) \leftarrow edge\_index$
$edge\_rep \leftarrow \text{concat}(X[start], X[end])$ ▷ edge representation
$weights \leftarrow \text{linear}(edge\_rep)$ ▷ edge weights
$weights_{\text{detach}} \leftarrow weights.\text{detach}()$ ▷ detached edge weights
$ranking \leftarrow \text{sorting}(weights_{\text{detach}})$ ▷ edge index sorted by descending detached edge weights (non-differentiable)
$index\_top\_r \leftarrow ranking[:k]$ ▷ select top_r edges based on ranking
$edge\_index\_top\_r \leftarrow edge\_index[:, index\_top\_r]$ ▷ (differentiable)
return $subgraph(edge\_index\_top\_r, X, edge\_index)$

Then we use EApooling to extract the causal subgraphs from the PrVC. Algorithm 2 illustrates the implementation of EApooling for extracting causal subgraphs of vulnerability. The algorithm takes node features and an edge index as input (Line 1) and outputs the causal subgraphs of the vulnerability (Line 2). It first retrieves the edge endpoints from the edge index (Line 3) and constructs an edge representation by concatenating the corresponding endpoint features (Line 4). A linear layer is then applied to compute a causal weight for each edge (Line 5).

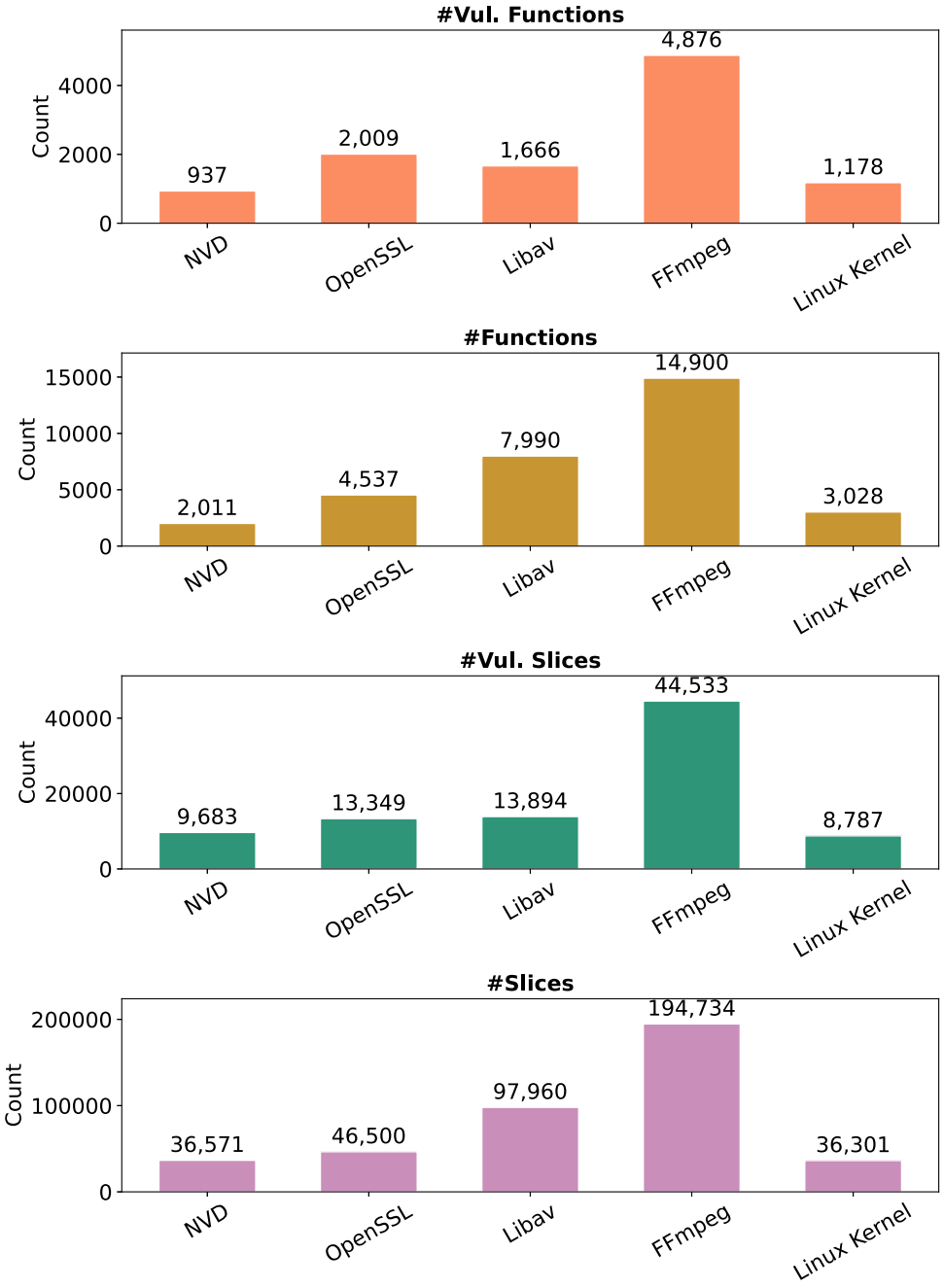

Fig. 6. Statistics on datasets.

A detached copy of these weights is created solely for ranking purposes (Lines 6-7). The algorithm next selects the top-r edges according to their ranked causal weights (Line 8) and extracts the corresponding edge index from the original edge index (Line 9). Finally, the induced subgraph formed by these selected edges is returned as the causal subgraphs of the vulnerability (Line 10).

It is worth noting that although the sorting operation in Line 7 is non-differentiable, EApooling preserves gradient flow by performing the ranking on a detached copy of the edge weights. This step is used solely to determine the top-r indices and does not participate in backpropagation. In Lines 8-9, these ranked indices are used to gather the corresponding edges from the original edge index, a differentiable indexing operation. Consequently, gradients from the loss can still propagate through the selected edges to update the parameters of the linear layer and the upstream encoder.

4.3. Vulnerability detection

In the vulnerability detection stage, we pass the causal subgraphs of vulnerability into another DIFFormer to update the node features again. This is because node features in the causal subgraphs are currently based on the structure and semantics of the PrVC, whereas the structure and semantics of the causal subgraphs are very different from those of the PrVC. Therefore, it is necessary to update the node features based on the structural relationships and semantic affinities between the nodes in the causal subgraphs. Additionally, since EApooling calculates edge attention and selects edges to retain based on a predefined pooling rate, some noisy nodes may remain in the extracted subgraphs. These noisy nodes can negatively impact the accuracy of subsequent vulnerability detection. DIFFormer's energy-constrained diffusion mechanism addresses both of these issues. First, it updates the node features based

on the structural relationships and semantic affinities between the nodes in the causal subgraphs. Then, during this process, it maintains low semantic affinity between noisy and non-noisy nodes, effectively limiting the spread of noisy node features to non-noisy nodes, thereby alleviating the impact of the remaining noisy nodes on the detection results. Finally, we pass the causal subgraphs of vulnerability into the classification module. The module performs additive pooling over all node features Z, followed by a linear transformation to obtain the final prediction:

$$\hat{y} = \mathbf{W} \cdot \texttt{AddPool}(Z) + b, \tag{6}$$

where $\texttt{AddPool}(Z) = \sum_{i=1}^{n} \mathbf{z}_i$, and $\mathbf{W}$ and b are learnable parameters. The output $\hat{y}$ denotes the vulnerability score of the input PrVC. If the vulnerability score $\hat{y}$ is greater than 0.5, the input PrVC is considered to contain a vulnerability. Otherwise, it is considered not to contain a vulnerability. To train the overall model, we adopt the binary cross-entropy loss with logits as the objective function. Given the vulnerability score $\hat{y}$ and the ground-truth label $y \in \{0, 1\}$, the loss is defined as:

$$\mathcal{L}_{\text{BCE}} = -\big[y \cdot \log \sigma(\hat{y}) + (1 - y) \cdot \log(1 - \sigma(\hat{y}))\big], \tag{7}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function.

5. Experiments

5.1. Experimental setup

5.1.1. Research questions

To evaluate the effectiveness of CIV, our experiments focus on answering the following research questions (RQs):

- **RQ1:** How effective is CIV compared to state-of-the-art vulnerability detection approaches?
- **RQ2:** How effective is DIFFormer compared to existing graph encoders in vulnerability detection?
- **RQ3:** How effective is EApooling compared to existing pooling methods in enhancing the extraction of causal subgraphs?
- **RQ4:** What is the impact of each part of CIV on its overall detection performance?
- **RQ5:** How does CIV perform under strict dataset splits and cross-project scenarios?

Implementation. In order to answer these questions, we implement CIV in Python using Pytorch[4]. The experiments are conducted on a Linux server with NVIDIA GeForce RTX 3090 GPU and Intel(R) Xeon(R) Silver 4214R CPU running at 2.40 GHz. The primary hyperparameters for training CIV are as follows: the AdamW optimizer with a learning rate 1e-3 is used for training the model up to 200 epochs. The number of DIFFormers is 2. We use a residual connection to link the node features before and after being updated by DIFFormer, with a residual weight of 0.5. The pooling ratio of EApooling may need to be adjusted across different datasets to achieve optimal performance. We set the pooling ratio to 0.6 for the Linux Kernel project and 0.7 for all other datasets.

Evaluation Metrics. In this paper, we use the following metrics to evaluate the effectiveness of our model: accuracy (ACC), precision (P), F1 score (F1), false positive rate (FPR), and false negative rate (FNR), recall rate (RECALL).

5.1.2. Dataset

In this work, we evaluate our method on two real-world vulnerability datasets: the publicly available NVD[5] dataset and the GITA dataset proposed by GraphFVD (Shao et al., 2025). The NVD dataset has been extensively used in previous vulnerability detection research (Cao et al., 2022; Li et al., 2021a, 2018), while the GITA dataset is directly adopted from the official version released by the authors of GraphFVD. GraphFVD selects four open-source projects: OpenSSL, Libav, FFmpeg, and Linux Kernel to build the GITA dataset. For the Linux Kernel, they first identify each vulnerability's GitHub link from the CVE database using its CVE ID, and then retrieve both the vulnerable code and the corresponding patch code. For the other three projects, they download source code associated with vulnerability by filtering and labeling commit messages on GitHub as vulnerability-fixing or non-vulnerability-fixing. Vulnerability-related functions are then extracted from these labeled commits. Each data sample in both datasets corresponds to a single function. For each function, we first construct its CPG. Then, we extract one or more PrVCs from the CPG. Each extracted PrVC is assigned an individual label indicating whether it is vulnerable. Specifically, if at least one node in the PrVC corresponds to a known vulnerable statement in the source code, the PrVC is labeled as vulnerable; otherwise, it is considered non-vulnerable.

Table 1
Distribution of function lengths (%) in NVD and GITA datasets.

Dataset	<200	200–400	400–600	600–800	800–1000	>1000
NVD	40.7	18.6	9.7	5.8	5.9	19.4
GITA	30.6	25.2	17.8	11.6	8.2	6.7

We present the statistics of the datasets used in our experiments in Fig. 6. The top-left subfigure shows the number of vulnerable functions in each project, while the top-right subfigure reports the total number of functions. Similarly, the bottom-left subfigure displays the number of vulnerable slices, and the bottom-right subfigure shows the total number of slices. Overall, the datasets include 10,666 vulnerable functions and 32,466 total functions, along with 90,246 vulnerable slices out of 412,066 total slices. Additionally, we provide the distribution of function lengths in both datasets, as shown in Table 1. These statistics demonstrate the diversity and scale of the datasets used for training and evaluating our model. For model training and evaluation, we randomly shuffled each project and allocated 80 % of the slices for training, reserving the remaining 20 % for validation and testing.

5.1.3. Baseline methods

To comprehensively assess the effectiveness of our proposed method, we compare it against two groups of baseline approaches: previous vulnerability detection methods and LLMs for code understanding. We directly use the baseline results reported in the GraphFVD (Shao et al., 2025).

The first group is previous vulnerability detection methods. SySeVR (Li et al., 2021b) and VulDeeLocator (Li et al., 2021a) are models that employ bidirectional gated recurrent unit (BGRU) to utilize sequence-level slices extracted from the PDG for vulnerability detection, focusing on syntax and intermediate representation (IR) semantics, respectively. CNN (Russell et al., 2018) transforms code into token sequences for feature extraction via convolutional neural networks. Devign (Zhou et al., 2019) employs a gated graph recurrent network (GGRN) on CPG to capture function-level semantics. Flawfinder[6] is a traditional rule-based tool that uses manually crafted rules to scan C/C++ code for known patterns. VulDeePecker (Li et al., 2018) identifies vulnerabilities associated with API usage by extracting code gadgets from PDG. MVD (Cao et al., 2022) and SnapVuln (Wu et al., 2023a) both rely on graph-level slices, using graph neural network (GNN) to encode control and data flow information. ReGVD (Nguyen et al., 2022) constructs token-based graphs from function-level code. VulChecker (Mirsky et al., 2023) builds an enhanced PDG (ePDG) from LLVM IR and applies Structure2Vec (Dai et al., 2016) for learning embeddings. GRACE (Lu et al., 2024) leverages in-context learning via LLMs with retrieved demonstrations. VulSim (Shimmi et al., 2024) fuses syntactic, semantic, and con-

[4] https://pytorch.org/
[5] https://nvd.nist.gov/
[6] https://dwheeler.com/flawfinder/

Table 2
Comparison of CIV and baseline methods on the NVD dataset (metrics: Accuracy, F1, and Precision).

Type	Approach	ACC(%)	F1(%)	P(%)
Baseline methods	Flawfinder	59.27	59.91	55.33
	CNN	68.49	68.80	74.87
	VulDeePecker	71.51	62.77	52.35
	MVD	74.00	65.20	61.50
	Devign	74.33	73.07	–
	EnGSP2F	–	76.60	76.60
	ReGVD	78.11	78.43	74.07
	VulDeeLocator	83.90	80.00	81.80
	SySeVR	87.14	78.72	81.99
	GraphFVD	91.11	82.92	82.27
LLM	ChatGPT-3.5-Turbo-1106	53.23	58.41	49.25
	DeepSeek-Coder-V2	71.14	68.13	59.05
	GPT-4-0613	85.43	81.76	**90.28**
Ours	CIV	**93.63**	**87.30**	86.36
	CIV_noise	91.66±0.49	82.47±1.02	86.18±1.25
	CIV_shuffle	92.15±1.15	83.76±2.71	85.76±1.01

Table 3
Comparison of CIV and baseline methods on the NVD dataset (metrics: FNR and FPR).

Type	Approach	FNR(%)	FPR(%)
Baseline methods	Flawfinder	34.69	46.00
	CNN	36.82	25.95
	VulDeePecker	21.68	31.54
	MVD	30.63	23.50
	Devign	26.00	–
	EnGSP2F	23.40	–
	ReGVD	17.71	27.10
	VulDeeLocator	21.70	12.10
	SySeVR	20.81	8.52
	GraphFVD	16.53	6.30
LLM	ChatGPT-3.5-Turbo-1106	29.35	62.16
	DeepSeek-Coder-V2	19.48	34.52
	GPT-4-0613	26.44	7.02
Ours	CIV	**11.73**	**4.59**
	CIV_noise	20.93±1.00	4.78±0.40
	CIV_shuffle	18.10±4.33	5.48±0.19

textual representations from multiple pre-trained models. EnGS2F (Xiao et al., 2024) creates a CRG from the PDG and predicts vulnerability using a GGNN.

The second group includes three high-performing LLMs for code: ChatGPT-3.5-Turbo-1106,[7] GPT-4-0613,[8] and DeepSeek-Coder-V2.[9] ChatGPT-3.5 and GPT-4, developed by OpenAI, are general-purpose models that support both natural and programming languages. DeepSeek-Coder-V2 is an open-source code-specific LLM trained from scratch on 2 trillion tokens and has shown superior performance on various coding benchmarks.

5.2. Experimental results

5.2.1. Experiments for answering RQ1

To demonstrate the effectiveness of CIV, we conducted comprehensive binary classification experiments on two real-world datasets: NVD and GITA. For function-level approaches, we performed vulnerability detection at the function granularity, treating each function as an independent instance and determining whether it contains a vulnerability. For slice-level approaches, including our proposed CIV, we performed binary classification at the slice granularity.

On the GITA dataset, four representative projects (OpenSSL, Libav, FFmpeg, and Linux Kernel) were evaluated independently, and the average of their results was reported as the final performance. For a fair comparison, the baseline performance results were directly cited from GraphFVD (Shao et al., 2025).

Tables 2–4 report the performance of all methods, where the optimal results are emphasized in bold. CIV consistently outperforms all baseline methods across nearly all evaluation metrics. Notably, compared to Flawfinder, CIV achieves significantly lower false negative rate (11.73 % vs. 34.69 % on NVD and 18.65 % vs. 57.81 % on GITA) and false positive rate (4.59 % vs. 46.00 % on NVD and 4.61 % vs. 41.32 % on GITA). This substantial performance gap highlights the inherent limitations of handcrafted detection rules, which struggle to generalize in the face of diverse and evolving vulnerability patterns across software projects and versions.

With the advantage of automatic feature extraction, deep learning methods such as CNN and ReGVD significantly surpass the rule-based approach. However, CNN and ReGVD treat source code as flat token sequences without explicitly modeling structural and semantic dependencies. This limits their capacity to learn complex vulnerability semantics. For instance, Devign utilizes GNN to encode CPG and achieves 74.33 % accuracy and 73.07 % F1 on the NVD dataset. By comparison, CIV improves these metrics to 93.63 % and 87.30 %, respectively—a substantial gain of 19.30 % and 14.23 %. Although Devign adopts a graph-based model, it does not reduce noise, leading to the retention of substantial noise and severely restricting its detection performance.

VulSim integrates syntactic, semantic, and contextual information from multiple pre-trained models but relies on decision trees for final classification. Although it performs relatively strongly on the GITA dataset, its high false positive rate and poor generalization limit its practical utility. CIV improves the F1 score by 3.14 % (82.39 % vs. 79.25 %) over VulSim and dramatically reduces the false positive rate from 35.70 % to 4.61 %, showcasing its superior robustness. EnGS2F constructs a CRG from the PDG and employs GGNN to learn representations. However, its graph construction introduces excessive redundant edges, which dilute the vulnerability-relevant semantics. CIV effectively addresses this issue and surpasses EnGS2F by 10.70 % in F1 score on the NVD dataset.

Compared to prior slice-based methods, CIV consistently delivers superior performance across datasets. Although VulDeePecker, SySeVR, and VulDeeLocator utilize slices to reduce noise, they primarily rely on sequence-level models and still suffer limited semantic coverage. VulDeePecker focuses narrowly on API call contexts and ignores key control and data dependencies. SySeVR and VulDeeLocator partially alleviate this by introducing PDG-based slicing and BGRU encoders, but their sequential architectures remain insufficient to capture comprehensive vulnerability semantics. By contrast, CIV extracts causal subgraphs of vulnerability and uses DIFFormer to learn their representations effectively. As a result, CIV reduces the false positive rate from 8.52 % (SySeVR) to 4.59 % and improves the F1 score by 8.58 % on the NVD dataset. Furthermore, although more advanced in slicing algorithm, methods such as MVD, SnapVuln, and GraphFVD still introduce too many noisy nodes, which dilute the vulnerability semantics required for precise detection. In contrast, CIV removes noise effectively by extracting causal subgraphs. Empirically, CIV achieves an F1 score improvement of 22.10 % over MVD and reduces the false negative rate from 30.63 % to 11.73 % on the NVD dataset. On the GITA dataset, CIV outperforms SnapVuln by 1.69 % in F1 score and significantly reduces the false positive rate from 20.41 % to 4.61 %. Compared with GraphFVD, CIV achieves a higher F1 score (82.39 % vs. 80.83 %) and further reduces false negatives (18.65 % vs. 20.49 %).

We also compared CIV with GRACE, a state-of-the-art LLM-based vulnerability detection method. As shown in Table 4, CIV achieves a significantly higher F1 score (82.39 % vs. 66.38 %) on the GITA dataset, demonstrating a performance gain of 16.01 %. Moreover, we evaluated three widely used LLMs—ChatGPT-3.5, GPT-4, and DeepSeek-Coder—

[7] https://openai.com/blog/chatgpt
[8] https://openai.com/research/gpt-4
[9] https://chat.deepseek.com

Table 4
Comparison of CIV and baseline methods on the GITA dataset (metrics unit: %).

Type	Approach	ACC(%)	F1(%)	P(%)	FNR(%)	FPR(%)
Baseline methods	Flawfinder	53.57	41.80	41.42	57.81	41.32
	CNN	58.93	50.76	48.82	46.53	40.77
	GRACE	60.13	66.38	54.58	**15.32**	61.19
	ReGVD	69.23	58.33	63.32	45.98	20.74
	Devign	72.26	73.26	–	–	–
	VulSim	75.34	79.25	75.00	16.00	35.70
	SnapVuln	80.07	80.70	80.10	18.40	20.41
	VulDeePecker	83.41	63.85	56.89	27.30	16.53
	SySeVR	87.23	75.92	77.45	25.56	8.20
	GraphFVD	92.05	80.83	82.18	20.49	**4.61**
LLM	ChatGPT-3.5-Turbo-1106	52.86	52.86	45.80	38.54	54.14
	DeepSeek-Coder-V2	63.87	60.39	56.82	35.57	36.54
	GPT-4-0613	72.25	67.02	71.11	36.63	20.63
Ours	CIV	**92.39**	**82.39**	**83.47**	18.65	**4.61**

Table 5
p-values from the Wilcoxon signed-rank test comparing CIV with GraphFVD and SySeVR across five projects.

Comparison	NVD	Linux	OpenSSL	Libav	FFmpeg
GraphFVD	$1.04e-27$	$2.35e-17$	$3.05e-09$	$4.37e-59$	$1.42e-37$
SySeVR	$2.95e-50$	$1.23e-29$	$9.15e-12$	$4.28e-74$	$1.10e-40$

Table 6
Comparison of DIFFormer with different graph encoders on NVD dataset.

Model	ACC(%)	F1(%)	P(%)	FPR(%)	FNR(%)
CIV	0.9363	0.8731	0.8636	0.0460	0.1173
CIV_gcn	0.9245	0.8410	0.8806	0.0360	0.1951
CIV_gat	0.9000	0.8105	0.7650	0.0874	0.1382
CIV_gps	0.8335	0.6600	0.6687	0.1065	0.3484

and observed that CIV consistently outperforms them across all evaluation metrics. Compared to the best-performing LLM, GPT-4, CIV improves accuracy, F1 score, and precision by 20.14 %, 15.37 %, and 12.36 %, respectively. Although LLMs exhibit strong generalization capabilities, they lack targeted supervision and do not perform noise reduction. Their frozen parameters and reliance on general-purpose pretraining limit their ability to capture domain-specific vulnerability patterns.

To prove the causality of the extracted causal subgraph of vulnerability, we conducted an interventional experiment in the NVD dataset. Specifically, we first extracted the causal subgraphs of vulnerability and the corresponding non-causal subgraphs from each PrVC. Then we intervened on the non-causal subgraphs S while keeping the causal subgraphs C unchanged. Two variants were designed: CIV_noise and CIV_shuffle. In CIV_noise, gaussian noise sampled from a normal distribution was added to the node features within the non-causal subgraphs, introducing random perturbations. In CIV_shuffle, the node features of non-causal subgraphs were randomly shuffled to destroy their semantic consistency. After applying these interventions, the modified PrVCs—consisting of both perturbed non-causal subgraphs and unaltered causal subgraphs—were fed into the full CIV inference pipeline for prediction. Each variant was evaluated five times with different random seeds, and the results are reported as the mean ± standard deviation. As shown in Table 2, both interventions led to only minor performance degradation (F1 decreased by 4.8 % for CIV_noise and 3.5 % for CIV_shuffle) compared with the original CIV. These results indicate that the causal subgraphs extracted by CIV are indeed highly relevant to vulnerability semantics. Leveraging these causal subgraphs for vulnerability detection effectively reduces the influence of noise. This finding provides empirical support for the causal relationship between the causal subgraphs and the vulnerability semantics.

We further assessed whether the observed performance gains of CIV are statistically significant rather than arising from random fluctuations. A Wilcoxon signed-rank test was conducted between CIV and the two baselines, GraphFVD and SySeVR, across five projects (NVD, Linux, OpenSSL, Libav, and FFmpeg). As shown in Table 5, all *p*-values are well below the significance threshold of 0.05, confirming that the performance improvements achieved by CIV are statistically significant and robust.

In summary, the superior performance of CIV can be primarily attributed to its explicit focus on noise reduction rather than just its learning capability. By leveraging causal subgraphs extraction, CIV accurately identifies and isolates the actual vulnerability patterns, significantly reducing the impact of noise. The integration of the DIFFormer model, which incorporates an energy-constrained diffusion mechanism, further strengthens our approach by effectively updating node features and minimizing the influence of residual noise. As a result, CIV demonstrates high detection accuracy and robust generalization in vulnerability detection.

5.2.2. *Experiments for answering RQ2*

We evaluate the effectiveness of DIFFormer in vulnerability detection by conducting experiments on the NVD dataset, comparing the original CIV model with three variants. In these variants, the two DIFFormer components in CIV are replaced by other graph encoders, specifically GCN, graph attention network (GAT), and GPS (Rampášek et al., 2022), while keeping the rest of the architecture unchanged. As shown in Table 6, CIV achieves the highest accuracy (93.63 %) and F1 score (87.31 %), while maintaining a low false positive rate (4.60 %) and false negative rate (11.73 %). Compared to the GCN-based variant, CIV improves F1 by 3.21 % (87.31 % vs. 84.10 %) and reduces the false negative rate by 7.58 % (11.73 % vs. 19.51 %). Against the GAT-based variant, CIV shows a 6.81 % gain in F1 (87.31 % vs. 81.05 %). The GPS-based variant performs the worst across all metrics, with the lowest accuracy (83.35 %), the lowest F1 score (66.00 %), and the highest false negative rate (34.84 %).

The experimental results on the NVD dataset demonstrate that DIFFormer achieves the best performance among all compared models in terms of accuracy and F1 score. This confirms its strong capability to represent vulnerability-related features. The superiority of DIFFormer stems from its ability to model semantic dependencies and structural relationships through an energy-constrained diffusion mechanism. This mechanism enables the model to capture global and local information in the graph while reducing noise interference, leading to a deeper understanding of vulnerability semantics. In contrast, GCN aggregates information from a fixed local neighborhood and treats all neighbors equally. Although this conservative aggregation results in a relatively lower false

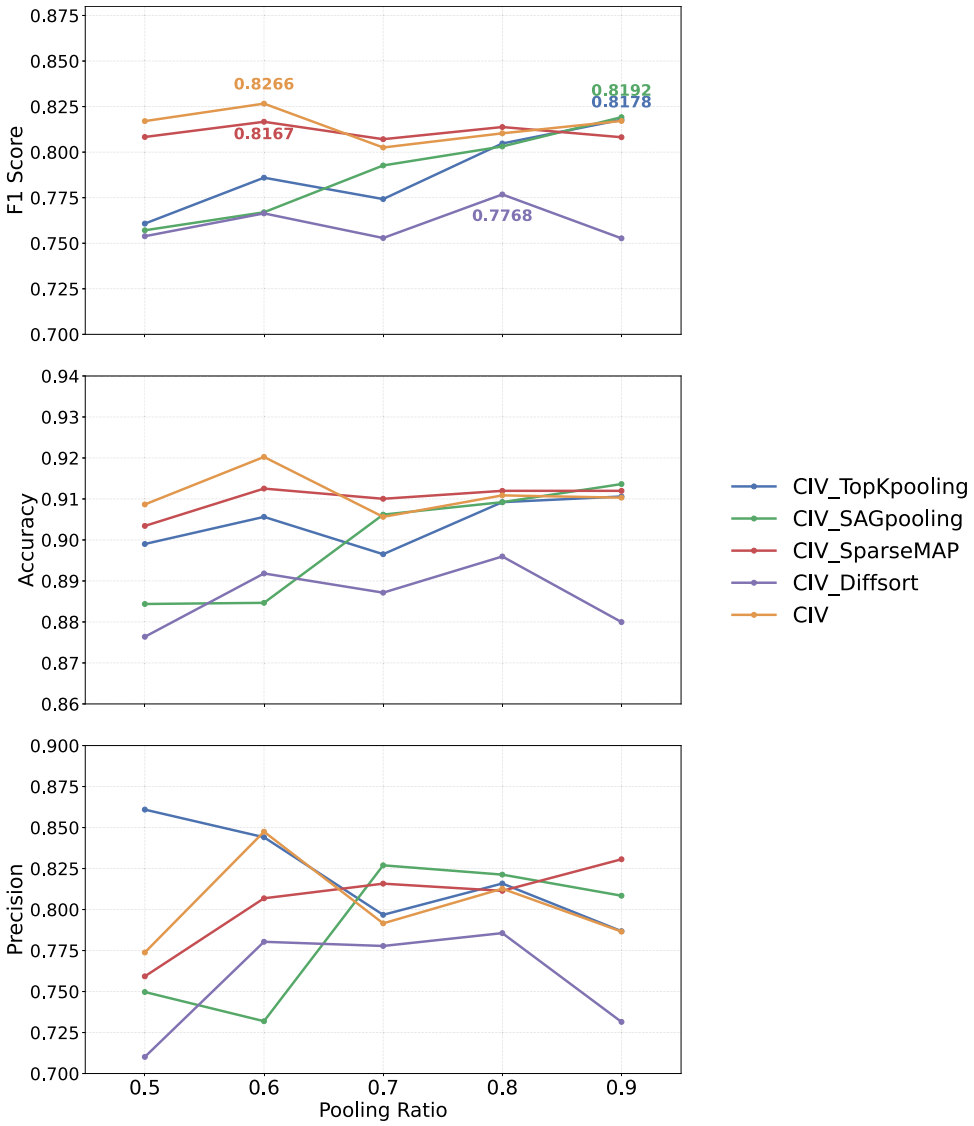

Fig. 7. Effectiveness comparison between EApooling and other pooling methods on the Linux project across different pooling ratios.

positive rate (3.60 %), it may miss subtle but critical vulnerability signals. GAT improves upon GCN by introducing attention over neighboring nodes, allowing the model to weigh them differently. However, GAT still focuses on a limited local region and lacks global context modeling, often misclassifying irrelevant patterns as vulnerability. Although GPS employs transformer-style global attention to capture long-range dependencies, it lacks step-wise information decay, making it prone to equally attending to semantically irrelevant nodes, leading to suboptimal performance in vulnerability detection tasks that demand precise semantic focus.

In summary, DIFFormer effectively integrates local structural information and global context. Its energy-constrained diffusion mechanism emphasizes semantically important nodes and reduces the interference from noisy nodes, enabling the model to detect complex vulnerability patterns with high accuracy and strong robustness.

5.2.3. *Experiments for answering RQ3*

To evaluate the effectiveness of the proposed EApooling layer, we construct four variants of the CIV model by replacing EApooling with other pooling mechods: CIV_TopKpooling, CIV_SAGpooling, CIV_SparseMAP, and CIV_Diffsort. Specifically, TopKpooling and SAGpooling are implemented using the official modules provided by PyTorch Geometric (Fey et al., 2025), while SparseMAP and differentiable sorting are integrated through open-source libraries. The CIV_SparseMAP variant leverages the LP-SparseMAP implementation (Niculae & Martins, 2020), and the differentiable sorting operator in CIV_Diffsort is implemented based on difftopk (Petersen et al., 2022). We conduct experiments on the Linux project across varying pooling ratios to comprehensively compare their performance.

In Fig. 7, CIV achieves its best performance at a pooling ratio of 0.6, where it attains an F1 score of 82.66 %, higher than the best F1 scores of the four variants. In contrast, CIV_TopKpooling and CIV_SAGpooling require larger retained subgraphs (ratio = 0.9) to reach their optimal F1 scores of 81.78 % and 81.92 %, respectively. CIV_SparseMAP attains the second-highest F1 score (81.66 %) at the same ratio 0.6, benefiting from its differentiable sparse selection mechanism, which softly enforces a top-k budget through a factor-graph optimization. This continuous relaxation produces stable gradients and smooth optimization. Still, the soft allocation of edge weights leads to partial inclusion of noise, limiting its ability to isolate strictly causal relations. By contrast, CIV_Diffsort shows a pronounced degradation (F1 = 77.68 %). This is mainly because differentiable sorting tends to assign nearly uniform importance scores to edges when their predicted weights are close, resulting in blurred distinctions between critical and irrelevant connections.

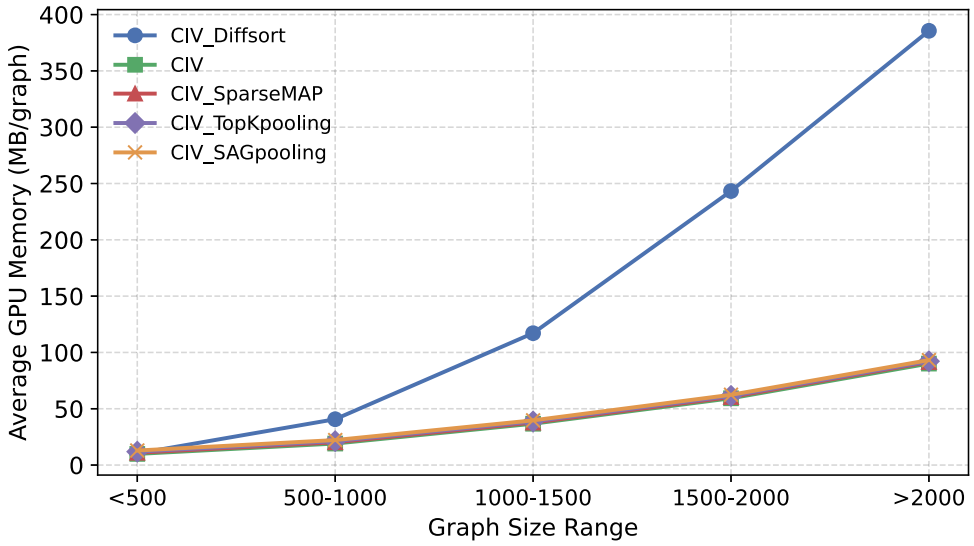

Fig. 8. Comparison of different pooling methods' average GPU memory consumption under various graph size ranges.

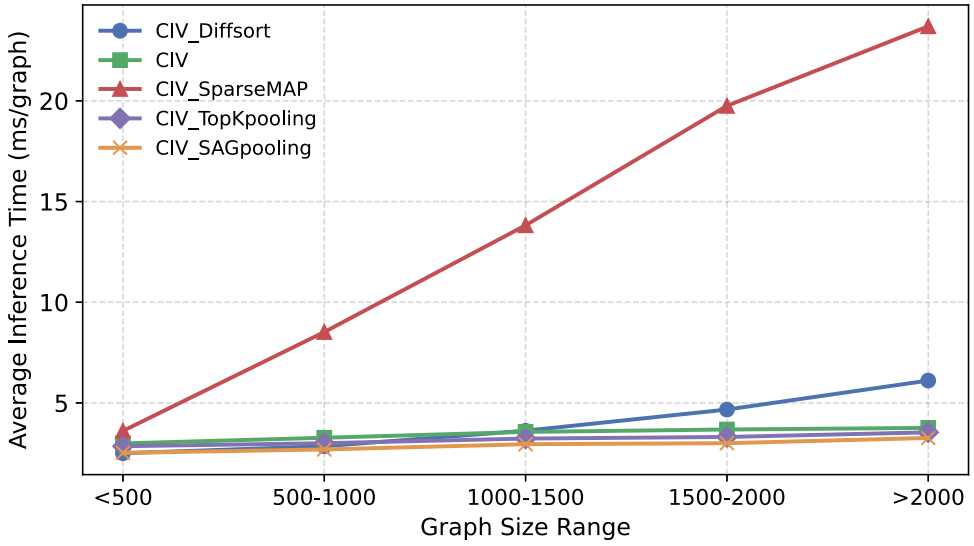

Fig. 9. Comparison of different pooling methods' average inference time under various graph size ranges.

Consequently, the extracted subgraphs fail to preserve clear semantic boundaries, weakening the representation of vulnerability-relevant structures.

Figs. 8 and 9 present the comparisons of average GPU memory consumption and inference time across different graph size ranges. CIV, CIV_TopKpooling, and CIV_SAGpooling exhibit comparable computational efficiency and memory usage, maintaining moderate resource consumption even for large graphs (>2000 nodes). In contrast, CIV_SparseMAP incurs notably longer inference time, while CIV_Diffsort shows substantially higher GPU memory consumption.

Overall, the results demonstrate that EApooling effectively extracts causal subgraphs of vulnerability, improving detection accuracy while maintaining comparable efficiency and memory usage to lightweight pooling methods.

5.2.4. *Experiments for answering RQ4*

To answer RQ4, we conduct an ablation study on the OpenSSL project by progressively removing key components from CIV to assess their contributions. As shown in Table 7, CIV (w/o EA) refers to the model where the EApooling layer is removed from CIV. CIV (w/o EA + RC) indicates that the EApooling layer and the residual connection are removed from CIV. CIV (w/o EA + RC + EDGE) means removing the EApooling layer, residual connection, and edge information from CIV. Finally, CIV (w/o EA + RC + SA) refers to the model where the EApooling layer, residual connection, and the calculation of semantic affinity between nodes in DIFFormer are removed, while the edge information is retained.

Table 7
Ablation study on the OpenSSL project.

Model	ACC(%)	P(%)	F1(%)	FPR(%)	FNR(%)
w/o EA	0.9056	0.8283	0.8428	0.0745	0.1421
w/o EA + RC	0.8991	0.8133	0.8333	0.0821	0.1458
w/o EA + RC + EDGE	0.7049	0.0000	0.0000	0.0000	1.0000
w/o EA + RC + SA	0.5610	0.3990	0.5644	0.6076	0.0364

As we progressively remove essential components of CIV, the overall performance across various metrics gradually declines. In CIV (w/o EA), the accuracy slightly drops to 0.9055, precision to 0.8282, and F1 score to 0.8428, compared to the complete model. This indicates that the EApooling layer plays an important role in noise reduction. CIV (w/o EA

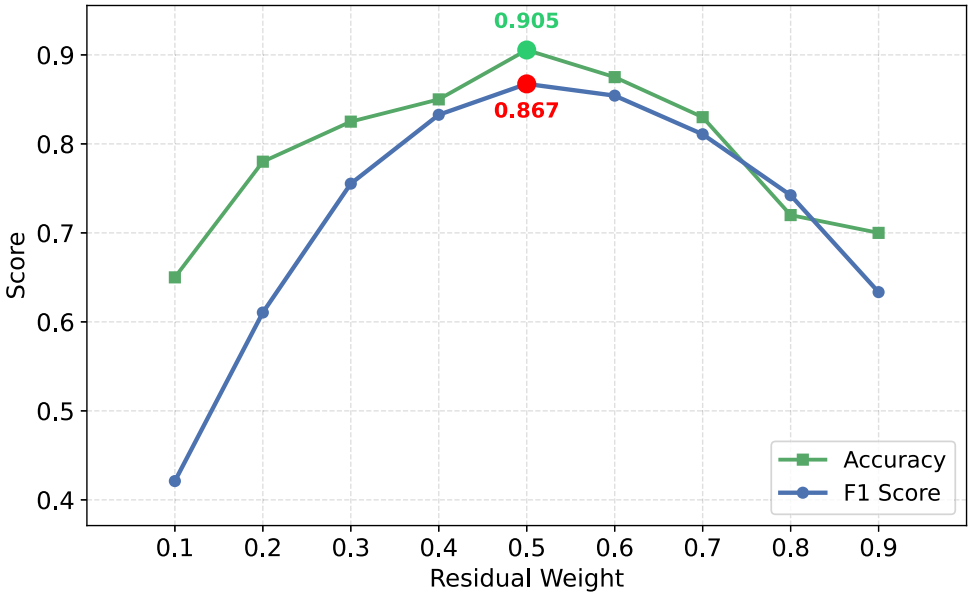

Fig. 10. Effect of residual weight parameters on F1 score and accuracy in the OpenSSL project.

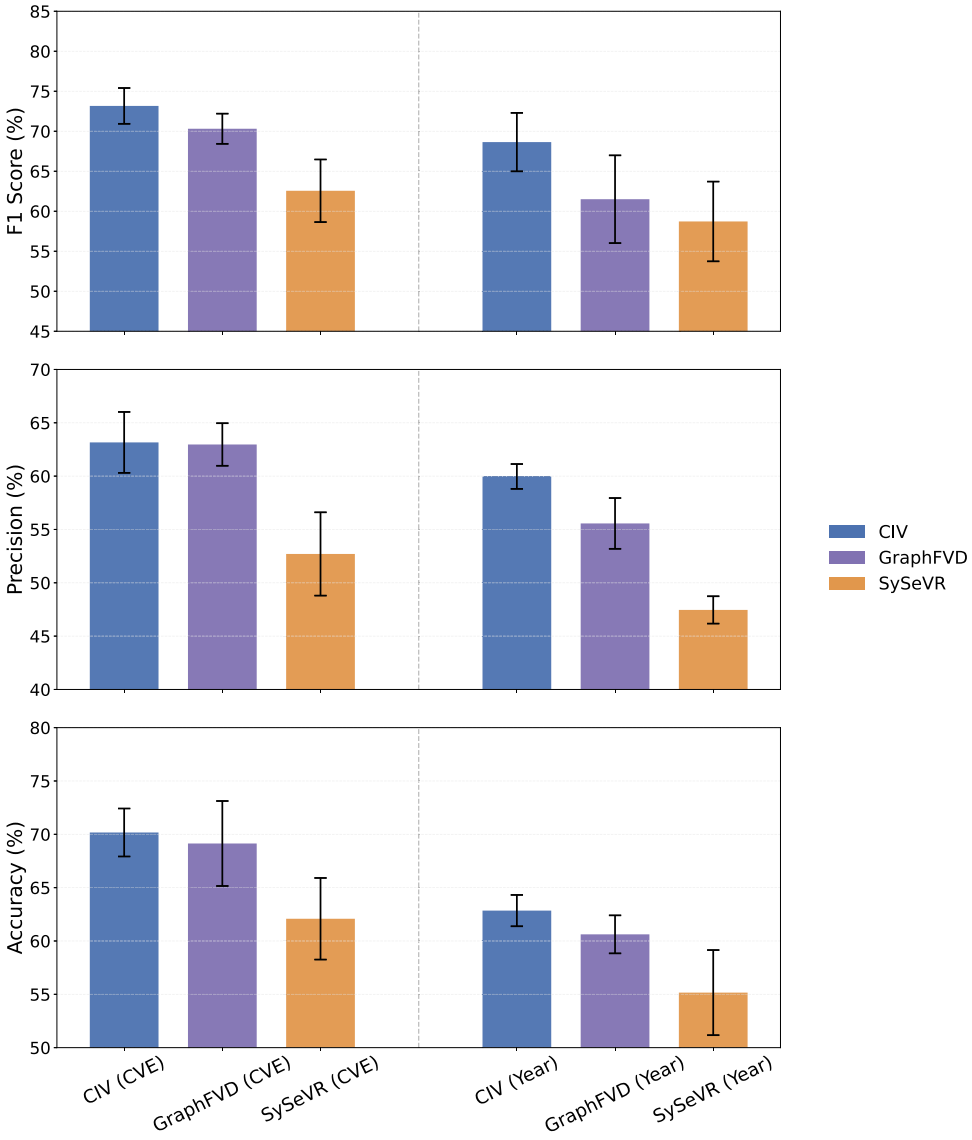

Fig. 11. CIV Performance under strict dataset splits (NVD Dataset).

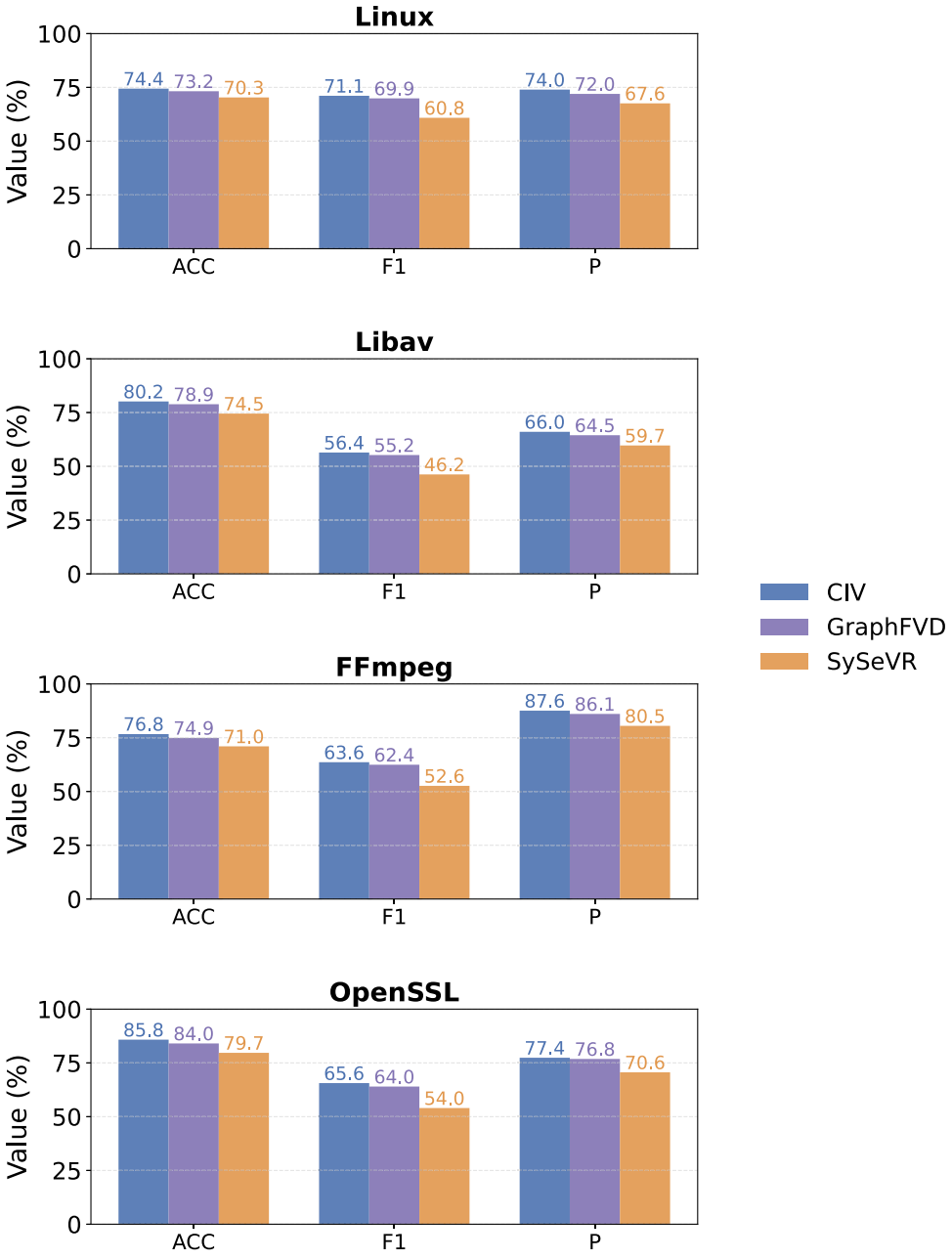

Fig. 12. Performance comparison under cross-project scenarios.

+ RC) shows further performance degradation, with accuracy decreasing to 0.8991, precision to 0.8133, and F1 score to 0.8332, suggesting that residual connection are critical for stabilizing feature learning across layers and preventing performance collapse. CIV (w/o EA + RC + EDGE) sees a drastic collapse in performance, with accuracy dropping sharply to 0.7048, and both precision and F1 score falling to 0. The FNR surges to 1.0000, indicating that the model fails to make correct predictions without structural edge information. Finally, in CIV (w/o EA + RC + SA), the accuracy drops to 0.5609, the precision to 0.3990, and the F1 score to 0.5643, while the FPR increases dramatically to 0.6075. The sharp rise in FPR, compared to CIV (w/o EA + RC), suggests that without the calculation of semantic affinity between nodes in DIFFormer, the model struggles to focus on causal subgraphs of vulnerability and is prone to misclassifying benign code as vulnerable.

Additionally, to analyze the impact of the residual weight, we varied the residual weight parameter from 0.1 to 0.9. Fig. 10 shows that the F1 score increases steadily as the residual weight rises and reaches its maximum (0.867) at 0.5, after which performance declines. A similar trend is observed for accuracy, which peaks at 0.905 when the residual weight is 0.5. This indicates that an appropriate residual weight (around 0.5) can effectively balance the influence between the original and residual feature pathways, enhancing detection accuracy while preventing overfitting caused by excessive residual contributions.

In summary, each component of the CIV model contributes meaningfully to its vulnerability detection capability. The EApooling layer and residual connection help refine semantic representations, while the edge information is essential for maintaining structural relationships. Most importantly, calculating semantic affinity between nodes in DIFFormer is crucial for guiding the model to focus on actual vulnerability patterns and reduce noise. These results confirm that these components in the CIV contribute meaningfully to its performance.

5.2.5. Experiments for answering RQ5

To answer RQ5, we designed two dataset splitting strategies to evaluate the robustness and generalization capability of CIV under more stringent evaluation conditions: split-by-CVE-ID and split-by-disclosure-year. In the split-by-CVE-ID setting, the dataset is divided into training, validation, and test subsets, and all samples associated with the same CVE ID are kept within the same subset. This prevents information leakage across samples derived from the same vulnerability. In the split-by-disclosure-year setting, the dataset is chronologically partitioned according to the vulnerability disclosure year, where the earliest 80 % of years are used for training and the remaining 20 % are randomly divided into validation and test sets. This setting requires the model to detect vulnerabilities disclosed in later years based on patterns learned from earlier data. For both splitting strategies, we performed five independent dataset splits using different random seeds.

As shown in Fig. 11, all three models experience a clear decline in performance when evaluated under stricter dataset partitioning strategies, particularly in the split-by-year setting. This observation is intuitive: splitting by CVE ID tends to preserve similar data distributions between the training and testing subsets, whereas chronological splitting introduces stronger distribution shifts-vulnerabilities disclosed in later years often exhibit new coding patterns or dependency structures unseen in earlier data. Such temporal shifts inherently increase the task difficulty and more accurately reflect real-world generalization challenges. Nevertheless, CIV consistently achieves the highest F1 scores across both scenarios, demonstrating better robustness and generalization capability.

Additionally, we evaluated the generalization capability of CIV under cross-project settings. Specifically, we trained CIV, GraphFVD, and SySeVR on the NVD dataset and tested their cross-project performance on four real-world projects: Linux, Libav, FFmpeg, and OpenSSL. This setup reflects a realistic yet challenging scenario where the model is required to identify vulnerabilities in previously unseen software systems. The cross-project setting is particularly difficult because the target projects differ substantially from the source training data regarding programming style, code structure, dependency composition, and vulnerability distribution.

Fig. 12 shows that all three models demonstrate generally low performance under the cross-project evaluation setting, indicating the difficulty of transferring learned representations to unseen projects. In particular, the performance degradation is more pronounced in the Libav project, where CIV achieves only a 56.4 % F1 score, highlighting the strong domain gap between training and target distributions. This result underscores that cross-project vulnerability detection remains a highly challenging task due to significant variations in code style, dependency structures, and vulnerability patterns across software systems.

6. Discussion

In this section, we discuss the limitations of CIV and outline potential directions for future improvement. CIV is a novel, causality-aware vulnerability detection framework that explicitly focuses on the code semantics responsible for vulnerability. Unlike existing slice-based methods, which often rely on fully manually defined slicing criteria, CIV introduces a causal perspective to identify the structural patterns that directly contribute to vulnerability formation. By combining the EApooling module for adaptive causal subgraph extraction and DIFFormer for noise suppression through energy-constrained diffusion, CIV effectively filters out spurious correlations and learns more robust vulnerability representations. This enables the model to focus on the actual causal mechanisms of vulnerability rather than superficial structural co-occurrences. Consequently, CIV achieves better generalization to unseen code. Despite its strengths, CIV still faces several challenges that warrant further investigation, particularly in improving subgraph extraction flexibility, integrating runtime information, and enhancing generalization under complex real-world conditions.

More flexible subgraphs extraction. Currently, we rely on the EApooling layer to identify causal subgraphs of vulnerability. While effective, this method is still influenced by the predefined pooling ratio, which limits the accuracy of the extracted subgraphs. To address this limitation, future work could focus on designing a more flexible subgraphs extraction module, allowing for more precise extraction of causal subgraphs.

Integration of runtime information. The complexity of modern C++ code often means that many vulnerabilities are only triggered at runtime. We may need to integrate further runtime information, such as memory states and I/O information, to detect these vulnerabilities. One possible avenue is combining our model with fuzzing techniques, which could help identify vulnerabilities that only manifest during specific execution scenarios, thereby increasing the range of vulnerability types that can be detected.

Limited generalization under strict dataset splits and cross-project settings. Our experiments in RQ5 demonstrate that CIV consistently outperforms GraphFVD and SySeVR across all three evaluation settings (split-by-CVE-ID, split-by-disclosure-year, and cross-project). This result confirms that CIV exhibits stronger robustness and better generalization capability under more stringent and realistic evaluation conditions. However, the overall performance in these scenarios remains limited, particularly under the split-by-year and cross-project settings, where the distribution shift between training and testing data is substantial. These findings indicate that while CIV effectively mitigates noise and captures transferable semantic patterns, future work should further enhance its adaptability to temporal and cross-domain variations, possibly through domain adaptation and continual learning strategies.

7. Conclusion

In this paper, we presented CIV, a causality-aware framework for source code vulnerability detection that mitigates the noise problem prevalent in deep learning-based approaches. Unlike prior slice-based methods that rely on fully manually defined slicing criteria, CIV introduces a novel slice representation—causal subgraphs of vulnerability—to capture the patterns that are related to vulnerability semantics. The proposed EApooling module can extract these causal subgraphs adaptively, while DIFFormer employs an energy-constrained diffusion mechanism to suppress the propagation of residual noise across nodes. This joint design enables CIV to focus on learning meaningful vulnerability features rather than spurious correlations. Experimental evaluations on real-world datasets demonstrate that CIV achieves higher accuracy and F1 scores than existing state-of-the-art methods. Future work should enhance CIV's adaptability to evolving vulnerability patterns and structural discrepancies, improving its robustness in complex, real-world projects.

CRediT authorship contribution statement

Zhenyu Gao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Visualization, Writing – original draft; **Lei Xiao:** Supervision, Methodology, Project administration, Writing – review & editing, Funding acquisition; **Xia Du:** Resources, Validation, Writing – review & editing; **Ying Xing:** Supervision, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Cao, S., Sun, X., Bo, L., Wu, R., Li, B., & Tao, C. (2022). MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks. In *Proceedings of the 44th international conference on software engineering* (pp. 1456–1468).

Cheng, B., Wang, K., Gao, C., Luo, X., Li, L., Guo, Y., Chen, X., & Wang, H. (2024). The vulnerability is in the details: Locating fine-grained information of vulnerable code identified by graph-based detectors. *CoRR*, abs/2401.02737.

Cheng, X., Zhang, G., Wang, H., & Sui, Y. (2022). Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis* (pp. 519–531).

Dai, H., Dai, B., & Song, L. (2016). Discriminative embeddings of latent variable models for structured data. In *International conference on machine learning* (pp. 2702–2711). PMLR.

Ding, H., Liu, Y., Piao, X., Song, H., & Ji, Z. (2025). Smartguard: An LLM-enhanced framework for smart contract vulnerability detection. *Expert Systems with Applications*, *269*, 126479.

Dong, Y., Tang, Y., Cheng, X., Yang, Y., & Wang, S. (2023). SedSVD: Statement-level software vulnerability detection based on relational graph convolutional network with subgraph embedding. *Information and Software Technology*, *158*, 107168.

Fey, M., Sunil, J., Nitta, A., Puri, R., Shah, M., Stojanovič, B., Bendias, R., Barghi, A., Kocijan, V., Zhang, Z. et al. (2025). Pyg 2.0: Scalable learning on real world graphs. *arXiv preprint arXiv:2507.16991*

Fu, M., & Tantithamthavorn, C. (2022). Linevul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th international conference on mining software repositories* (pp. 608–620).

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C. B., Drain, D., Sundaresan, N., Yin, J., Jiang, D., & Zhou, M. (2020). GraphcodeBERT: Pre-training code representations with data flow. *CoRR*, abs/2009.08366.

Guo, W., Fang, Y., Huang, C., Ou, H., Lin, C., & Guo, Y. (2022). Hyvuldect: A hybrid semantic vulnerability mining system based on graph neural network. *Computers & Security*, *121*, 102823.

Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., & Jin, H. (2021a). Vuldeelocator: A deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing*, *19*(4), 2821–2837.

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., & Chen, Z. (2021b). Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, *19*(4), 2244–2258.

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., & Zhong, Y. (2018). Vuldeepecker: A deep learning-based system for vulnerability detection. In *25th annual network and distributed system security symposium, NDSS 2018, San Diego, California, USA, February 18–21, 2018*. The Internet Society.

Lu, G., Ju, X., Chen, X., Pei, W., & Cai, Z. (2024). Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software*, *212*, 112031.

Luo, Y., Xu, W., & Xu, D. (2022). Compact abstract graphs for detecting code vulnerability with GNN models. In *Proceedings of the 38th annual computer security applications conference* (pp. 497–507).

Ma, X., He, Y., Keung, J., Tan, C., Ma, C., Hu, W., & Li, F. (2025). On the value of imbalance loss functions in enhancing deep learning-based vulnerability detection. *Expert Systems with Applications*, *291*, 128504.

Mechri, A., Ferrag, M. A., & Debbah, M. (2025). Secureqwen: Leveraging LLMs for vulnerability detection in python codebases. *Computers & Security*, *148*, 104151.

Mirsky, Y., Macon, G., Brown, M. D., Yagemann, C., Pruett, M., Downing, E., Mertoguno, J. S., & Lee, W. (2023). Vulchecker: Graph-based vulnerability localization in source code. In J. A. Calandrino, & C. Troncoso (Eds.), *32nd USENIX security symposium, USENIX security 2023, Anaheim, CA, USA, August 9–11, 2023* (pp. 6557–6574). USENIX Association.

Nguyen, V.-A., Nguyen, D. Q., Nguyen, V., Le, T., Tran, Q. H., & Phung, D. (2022). Regvd: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings* (pp. 178–182).

Niculae, V., & Martins, A. (2020). Lp-sparsemap: Differentiable relaxed optimization for sparse structured prediction. In *International conference on machine learning* (pp. 7348–7359).

Pang, Y., Liu, X., Huang, T., Hong, Y., Huang, J., Duan, S., & Dong, C. (2025). Graph-based contract sensing framework for smart contract vulnerability detection. *IEEE Transactions on Big Data*, *11*, 3356–3368.

Petersen, F., Kuehne, H., Borgelt, C., & Deussen, O. (2022). Differentiable top-k classification learning. In *International conference on machine learning* (pp. 17656–17668). PMLR.

Pornprasit, C., & Tantithamthavorn, C. K. (2022). Deeplinedp: Towards a deep learning approach for line-level defect prediction. *IEEE Transactions on Software Engineering*, *49*(1), 84–98.

Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., & Beaini, D. (2022). Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems*, *35*, 14501–14515.

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., & McConley, M. (2018). Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)* (pp. 757–762). IEEE.

Salimi, S., & Kharrazi, M. (2022). Vulslicer: Vulnerability detection through code slicing. *Journal of Systems and Software*, *193*, 111450.

Shao, M., Ding, Y., Cao, J., & Li, Y. (2025). GraphFVD: Property graph-based fine-grained vulnerability detection. *Computers & Security*, *151*, (p. 104350).

Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., & Huang, J. (2025). Llms in software security: A survey of vulnerability detection techniques and insights. *ACM Computing Surveys*. https://doi.org/10.1145/3769082

Shestov, A., Levichev, R., Mussabayev, R., Maslov, E., Zadorozhny, P., Cheshkov, A., Mussabayev, R., Toleu, A., Tolegen, G., & Krassovitskiy, A. (2025). Finetuning large language models for vulnerability detection. *IEEE Access*, *13*, 38889–38900 .

Shimmi, S., Rahman, A., Gadde, M., Okhravi, H., & Rahimi, M. (2024). {VulSim}: Leveraging similarity of {Multi-Dimensional} neighbor embeddings for vulnerability detection. In *33rd USENIX security symposium (USENIX security 24)* (pp. 1777–1794).

Steenhoek, B., Gao, H., & Le, W. (2024). Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th ieee/acm international conference on software engineering* (pp. 1–13).

Tang, M., Tang, W., Gui, Q., Hu, J., & Zhao, M. (2024). A vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN). *Expert Systems with Applications*, *238*, 122216.

Tian, Z., Tian, B., Lv, J., Chen, Y., & Chen, L. (2024). Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding. *Expert Systems with Applications*, *238*, 121865.

Wang, H., Ye, G., Tang, Z., Tan, S. H., Huang, S., Fang, D., Feng, Y., Bian, L., & Wang, Z. (2020). Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security*, *16*, 1943–1958.

Wang, W., Nguyen, T. N., Wang, S., Li, Y., Zhang, J., & Yadavally, A. (2023). DeepVD: Toward class-separation features for neural network vulnerability detection. In *2023 IEEE/ACM 45th international conference on software engineering (ICSE)* (pp. 2249–2261). IEEE.

Wattanakriengkrai, S., Thongtanunam, P., Tantithamthavorn, C., Hata, H., & Matsumoto, K. (2020). Predicting defective lines using a model-agnostic technique. *IEEE Transactions on Software Engineering*, *48*(5), 1480–1496.

Wu, B., Liu, S., Xiao, Y., Li, Z., Sun, J., & Lin, S.-W. (2023a). Learning program semantics for vulnerability detection via vulnerability-specific inter-procedural slicing. In *Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering* (pp. 1371–1383).

Wu, B., Zou, F., Yi, P., Wu, Y., & Zhang, L. (2023b). Slicedlocator: Code vulnerability locator based on sliced dependence graph. *Computers & Security*, *134*, 103469.

Wu, Q., Yang, C., Zhao, W., He, Y., Wipf, D., & Yan, J. (2023c). Difformer: Scalable (graph) transformers induced by energy constrained diffusion. *CoRR*, abs/2301.09474.

Wu, Y., Wang, X., Zhang, A., He, X., & Chua, T. (2022a). Discovering invariant rationales for graph neural networks. *CoRR*, abs/2201.12872.

Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., & Jin, H. (2022b). Vulcnn: An image-inspired scalable vulnerability detection system. In *Proceedings of the 44th international conference on software engineering* (pp. 2365–2376).

Xiao, P., Xiao, Q., Zhang, X., Wu, Y., & Yang, F. (2024). Vulnerability detection based on enhanced graph representation learning. *IEEE Transactions on Information Forensics and Security*, *19*, 5120–5135.

Yu, X., Lin, G., Hu, X., Keung, J. W., & Xia, X. (2025). Less is more: Unlocking semi-supervised deep learning for vulnerability detection. *ACM Transactions on Software Engineering and Methodology*, *34*(3), 1–37.

Yuan, B., Lu, Y., Fang, Y., Wu, Y., Zou, D., Li, Z., Li, Z., & Jin, H. (2023). Enhancing deep learning-based vulnerability detection by building behavior graph model. In *2023 IEEE/ACM 45th international conference on software engineering (ICSE)* (pp. 2262–2274). IEEE.

Zhang, C., Liu, H., Zeng, J., Yang, K., Li, Y., & Li, H. (2024). Prompt-enhanced software vulnerability detection using chatgpt. In *Proceedings of the 2024 IEEE/ACM 46th international conference on software engineering: Companion proceedings* (pp. 276–277).

Zhang, Y., Ju, X., Chen, X., Misbahul, A., & Ren, Z. (2025). Hgan4vd: Leveraging heterogeneous graph attention networks for enhanced vulnerability detection. *Computers & Security*, (p. 104548).

Zhou, X., Cao, S., Sun, X., & Lo, D. (2025). Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology*, *34*(5), 1–31.

Zhou, Y., Liu, S., Siow, J., Du, X., & Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in Neural Information Processing Systems*, *32*, 10197–10207.